

LLM Post-Training and High-Performance Serving

Kunling Geng

Aydin Abiar

Eric Tang



Kunling Geng

Architect-
LLM & Agentic AI

Anyscale



Aydin Abiar

DevEx Engineer-
Ray Serve LLM
Specialist

Anyscale



Eric Tang

Software Engineer -
LLMs

Anyscale

RAY SUMMIT

August 24 – 26 · San Francisco

RAY SUMMIT

By  Anyscale

● LLM Post Training & High Performance Serving

Join at
slido.com
#2260 872





Agenda

→ **Part1: Introduction**

- ◆ The Coding Agent Boom
- ◆ Scalable LLM Serving
- ◆ Building your LLM & Agent Flywheel
- ◆ Anyscale Agent Skills

→ **Part2: LLM High Performance Serving and Optimizations**

- ◆ Deploying an LLM for Coding Agents (4 Steps)
- ◆ Hands-on Lab: LLM Serving for Coding Agents
- ◆ LLM Serving Optimizations for Coding Agents
- ◆ Q&A + Break

→ **Part3: Scalable LLM & Agentic Post-training**

- ◆ Intro to Post-Training and RL
- ◆ Case Study: Post-Training a Terminal Agent
- ◆ Demo: Fully Async RL on SkyRL

Part 1: Introduction

The Coding Agent Boom



Codex



Antigravity



Cost of Coding Agents is Skyrocketing

TECH AI MICROSOFT

Microsoft starts canceling Claude Code licenses



/ Thousands of Microsoft developers will use GitHub C CLI instead



Marty Kausas
@marty_kausas



Our Anthropic bill is about to jump from \$400K → \$1.4M/yr.

Not because usage exploded, but because we're about to cross 150 seats.

Past 150 seats you're forced into Enterprise tier. Seats stop including any usage, every token bills at standard API rates. At our current run rate that's 3.5x overnight.

Unfiltered thoughts on AI spend:

EDITORS' PICK | INNOVATION > CLOUD

Uber Burns Its 2026 AI Budget In Four Months On Claude Code

By [Janakiram MSV](#), Senior Contributor. © I cover emerging technologies with ...

Follow Author

Published May 17, 2026, 11:18pm EDT, Updated May 27, 2026, 12:17pm EDT

GitHub Copilot Costs Skyrocket As Users Are Pushed to Per-Token Billing

The new system will charge users based on how much the AI does, rather than how many requests they make, and some users are finding their bills jumping by 10x or more.



By [Jon Martindale](#) June 2, 2026



Swapping the Coding Agents with Open LLMs

The agents are just clients speaking standard APIs. You can point them at **your** LLM endpoint:

Agent	Endpoints
Cursor	/v1/chat/completions (OpenAI)
Claude Code	/v1/messages (Anthropic)
Codex	/v1/responses (OpenAI)

Benefits:

- Reduce cost
- Avoid rate throttling
- Security and data control
- Avoid vendor lock-in
- Evolve the LLM with your own data
- Community and transparency

Scalable LLM Serving with Ray Serve LLM at Anyscale

- Enables autoscaling, load balancing
- Multi-LoRA model deployment
- Unified LLM Endpoint with multiple LLMs
- OpenAI & Anthropic - compatible API via Direct Streaming
- Prefix cache aware routing & custom routing
- Advanced serving patterns (PD disaggregation, etc)
- Built-in metrics, dashboards and observability

vLLM inference engine

- Continuous batching
- Memory management
- Structured output
- Tool calling
- Quantization support
- Optimized CUDA kernels

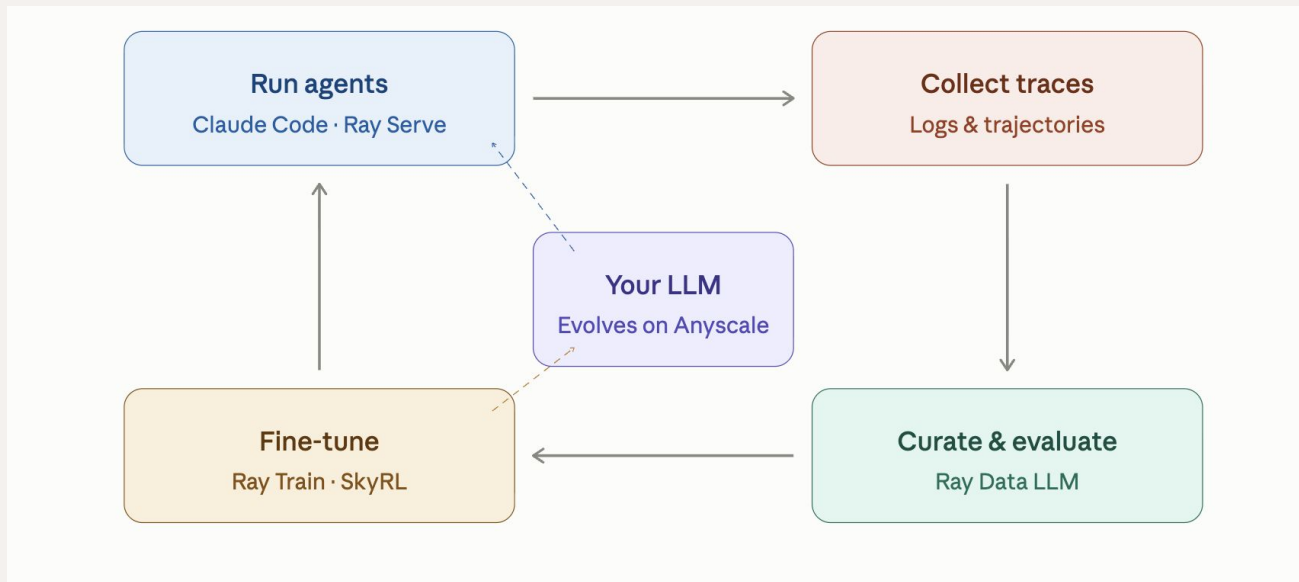
vLLM: Inference Engine

- + High throughput, low latency serving
- No autoscaling
- No multi-LoRA management

Ray Serve: Model Serving Library

- + Pythonic API for autoscaling
- + Model multiplexing
- + API for complex deployment pipelines

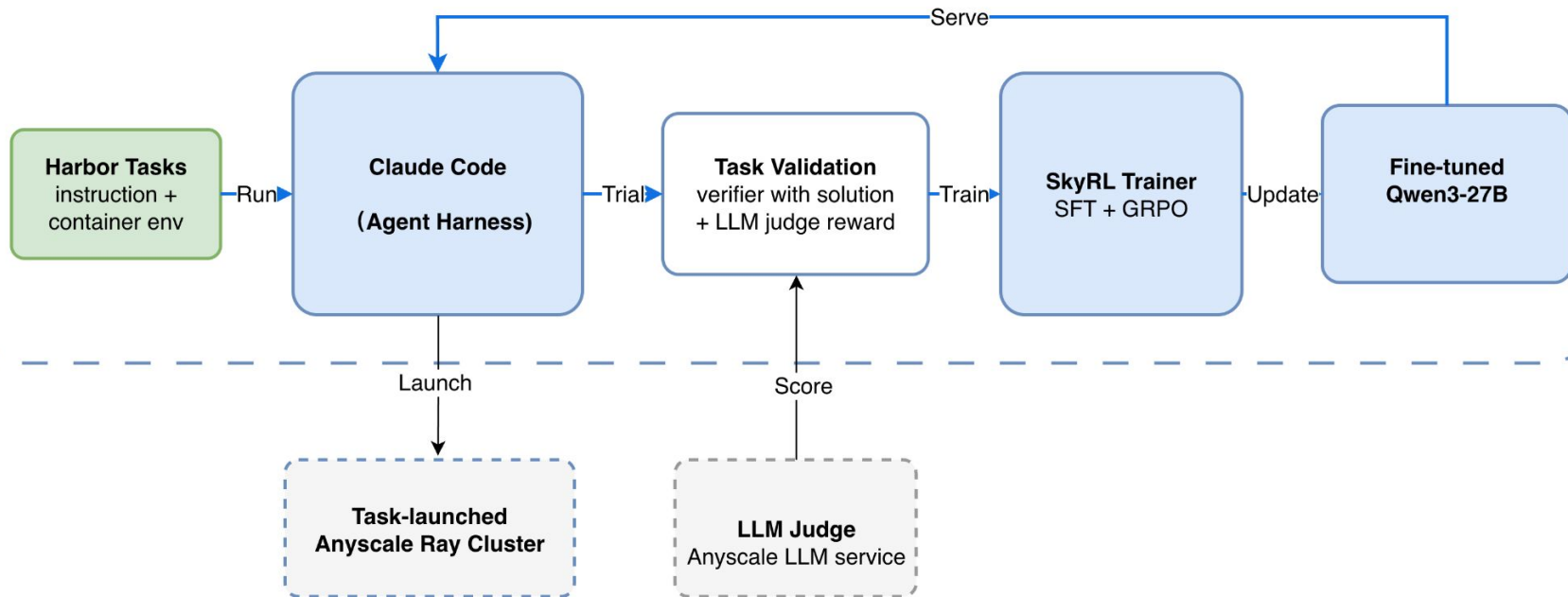
Building the AI Agent Flywheel



Powered by Anyscale Agent Skills

Fine-tune Qwen3 with SkyRL with Claude Code traces

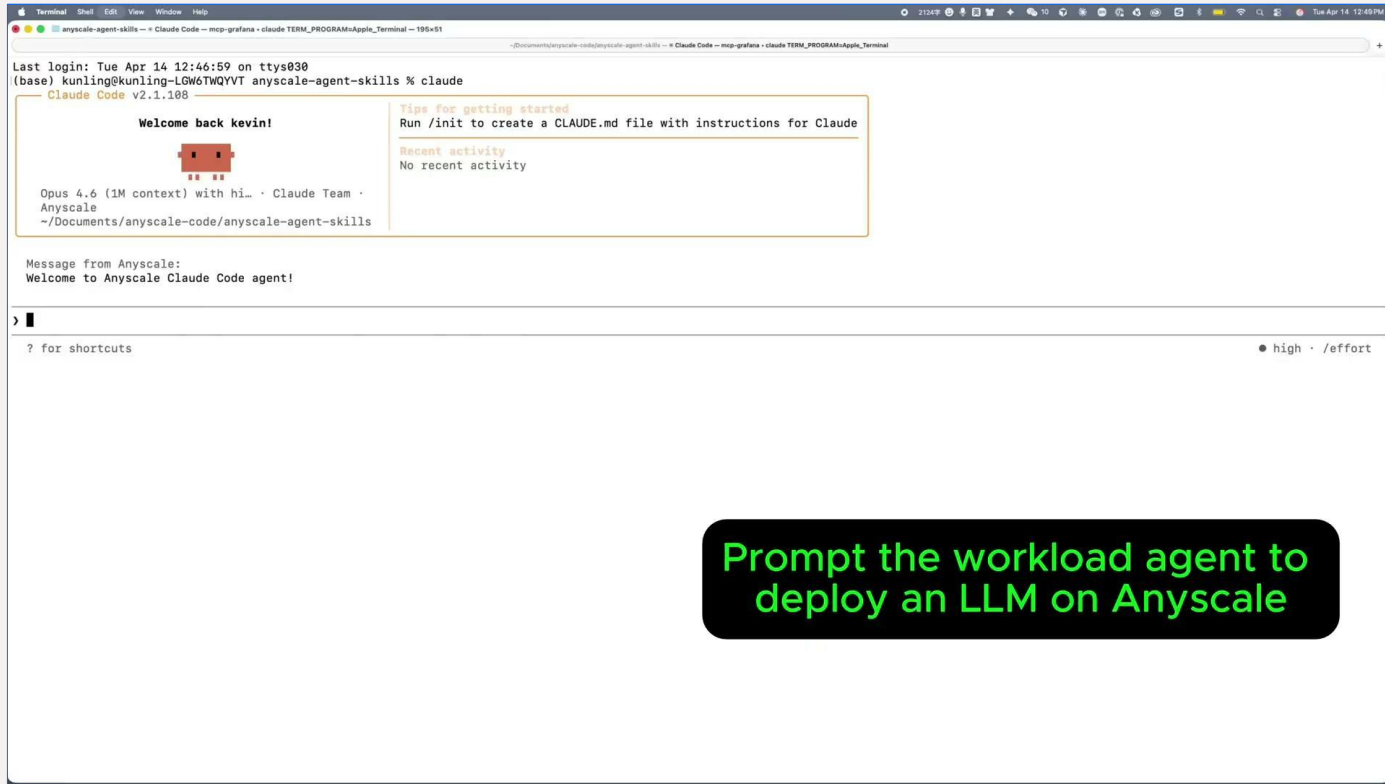
Harbor + SkyRL Trainer Workspace



LLMs are challenging: starting with hosting them

- **Model Diversity:** The sheer number of models (7B, 70B, 670B, Dense vs. MoE) each requires a unique and complex serving strategy.
- **Hardware Choice:** Choosing the right GPU and configuration with parallelisms (TP, PP, EP, DP) is a complex balance of cost, memory, and speed.
- **Configuration Hell:** The huge number of optimization "knobs" (e.g., quantization, parallelism, chunked prefill, P/D disaggregation) makes finding the optimal setup incredibly difficult.
- **Technique Overload:** Knowing *when and what* to use for a specific use case is a major challenge.
- **Constant Trade-offs:** You are forced to constantly balance four competing factors: latency, throughput, quality, and cost.

Anyscale Agent skills - LLM Serving



```

Terminal Shell Edit View Window Help
anyScale-agent-skills - Claude Code - mcp-grafana - Claude TERM_PROGRAM=Apple_Terminal - 195x51
Last login: Tue Apr 14 12:46:59 on ttys030
(base) kunling@kunling-LGW6TWQYVT anyscales-agent-skills % claude
Claude Code v2.1.108

Welcome back kevin!

Opus 4.6 (1M context) with hi... · Claude Team ·
Anyscale
~/Documents/anyscale-code/anyscale-agent-skills

Message from Anyscale:
Welcome to Anyscale Claude Code agent!

> █
? for shortcuts
● high · /effort
  
```

Prompt the workload agent to
deploy an LLM on Anyscale

Anyscale skills cover the full customer lifecycle.



GROUP 01

Workload

Code & config scaffolding.
Validates GPU memory.
Uses current APIs. Tested templates.

```
/anyscale-workload-llm-serving  
/anyscale-workload-llm-post-training  
/anyscale-workload-ray-serve  
/anyscale-workload-ray-data  
/anyscale-workload-ray-train  
/anyscale-workload-batch-embedding  
/anyscale-workload-physical-ai
```



GROUP 02

Platform

Inspect → diagnose → fix
→ redeploy → verify. All
inside the coding agent.

```
/anyscale-platform-inspect  
/anyscale-platform-run  
/anyscale-platform-fix  
/anyscale-platform-ask
```



GROUP 03

Infra

Guided Anyscale deploys
— Kubernetes, AWS, GCP.
Helm, Terraform, IAM.

```
/anyscale-infra-kubernetes  
/anyscale-infra-aws-vm  
/anyscale-infra-gcp-vm
```



GATED

GROUP 04

Optimization Services Program

Bottleneck analysis,
GPU-waste reduction,
production-readiness.
Validated across hundreds
of deployments.

AI agents paired with our forward deployed
engineers analyze throughput bottlenecks

<https://docs.anyscale.com/agent-skills>



Part 2: LLM High-Performance Serving or Coding Agents

The steps of deploying an LLM for coding agents

1. **LLM selection** — pick the model that fits quality + footprint
2. **Hardware selection** — GPU + parallelism (TP/PP)
3. **Set up Serving Deployment Configurations** — KV-cache math → concurrency → replicas
4. **Deploy on Anyscale & Connect your agents** — Claude Code / Codex / Cursor

Step 1: LLM Selection

Agent Arena Leaderboard (Aug 2026)

Rank ↕	Model	Net Improvement ↕	Confirmed Success ↕	Praise vs Complaint ↕	Steerability ↕
1 1 → 6	 Claude Opus 5 (High) Anthropic · Proprietary	▲ 12.47% ±1.54% 	▲ 15.41% ±3.16%	▲ 20.52% ±5.59%	▲ 11.15% ±2.98%
2 1 → 6	 Claude Opus 5 (Max) Anthropic · Proprietary	▲ 12.00% ±1.80% 	▲ 18.52% ±3.24%	▲ 19.13% ±6.56%	▲ 6.65% ±3.66%
3 1 → 6	 Claude Fable 5 (High) Anthropic · Proprietary	▲ 11.57% ±1.70% 	▲ 12.44% ±3.24%	▲ 21.37% ±5.95%	▲ 8.99% ±3.55%
4 1 → 6	 Kimi K3 (Max) Moonshot · Kimi K3 license	▲ 10.41% ±0.62% 	▲ 17.97% ±1.23%	▲ 18.31% ±2.23%	▲ 5.96% ±1.13%
5 1 → 11	 GPT 5.6 So1 (xHigh) OpenAI · Proprietary	▲ 9.74% ±1.39% 	▲ 10.00% ±2.87%	▲ 22.20% ±5.11%	▲ 5.77% ±2.88%
6 1 → 11	 Claude Opus 4.8 (High) Anthropic · Proprietary	▲ 9.55% ±1.51% 	▲ 8.26% ±2.70%	▲ 21.65% ±4.97%	▲ 9.01% ±2.86%
7 5 → 12	 GPT 5.5 (xHigh) OpenAI · Proprietary	▲ 8.51% ±0.93% 	▲ 4.42% ±1.96%	▲ 13.46% ±3.23%	▲ 8.86% ±1.76%
8 5 → 16	 Claude Opus 4.7 (High) Anthropic · Proprietary	▲ 8.12% ±1.24% 	▲ 5.62% ±2.53%	▲ 12.36% ±4.17%	▲ 8.44% ±2.47%
9 5 → 16	 GPT 5.5 (High) OpenAI · Proprietary	▲ 7.74% ±0.84% 	▲ 4.09% ±1.74%	▲ 11.26% ±2.94%	▲ 9.12% ±1.56%
10 5 → 17	 Claude Opus 4.7 Anthropic · Proprietary	▲ 7.40% ±1.25% 	▲ 4.99% ±2.59%	▲ 11.51% ±4.09%	▲ 9.32% ±2.45%
11 5 → 21	 Claude Sonnet 5 (High) Anthropic · Proprietary	▲ 6.62% ±2.19% 	▲ 1.51% ±4.60%	▲ 14.80% ±7.61%	▲ 4.63% ±4.65%
12 7 → 19	 Claude Opus 4.6 Anthropic · Proprietary	▲ 6.60% ±1.20% 	▲ 4.85% ±2.53%	▲ 8.49% ±3.90%	▲ 7.47% ±2.33%

- Frontier proprietary models still lead overall
- Open-weight models are now very competitive

<https://arena.ai/leaderboard/agent>

Leading Open-Weight Models LLMs for coding agents (Aug 2026)

Model	Architecture & Parameters	Notable for coding
Qwen 3.6, 3.8 Series	Dense + MoE family (27B, 35B, 2.4T)	Strong tool-calling, broad size range
DeepSeek V4 (Flash, Pro)	Large MoE (280B, 1.6T)	High reasoning + code, efficient active params
Kimi K3	Extremely Large MoE (2.8T)	Long-context, agentic workflows
GLM-5.2, 5.3	Large MoE (750B)	Agentic coding + tool use

Today's focus: Qwen 3.6-27B

	Qwen3.5-27B	Qwen3.5-397B-A17B	Gemma4-31B	Claude 4.5 Opus	Qwen3.6-35B-A3B	Qwen3.6-27B
Coding Agent						
SWE-bench Verified	75.0	76.2	52.0	80.9	73.4	77.2
SWE-bench Pro	51.2	50.9	35.7	57.1	49.5	53.5
SWE-bench Multilingual	69.3	69.3	51.7	77.5	67.2	71.3
Terminal-Bench 2.0	41.6	52.5	42.9	59.3	51.5	59.3
SkillsBench _{Avg5}	27.2	30.0	23.6	45.3	28.7	48.2
QwenWebBench	1068	1186	1197	1536	1397	1487
NL2Repo	27.3	32.2	15.5	43.2	29.4	36.2
Claw-Eval _{Avg}	64.3	70.7	48.5	76.6	68.7	72.4
Claw-Eval _{Pass^3}	46.2	48.1	25.0	59.6	50.0	60.6
QwenClawBench	52.2	51.8	41.7	52.3	52.6	53.4

Average: 61.31

59.12

Note: Average excludes QwenWebBench (different scale).

After today's session, try deploying Qwen3.8-27b which has even better performance

Step 2: Hardware Selection

GPU Selection Guide

General Guidelines

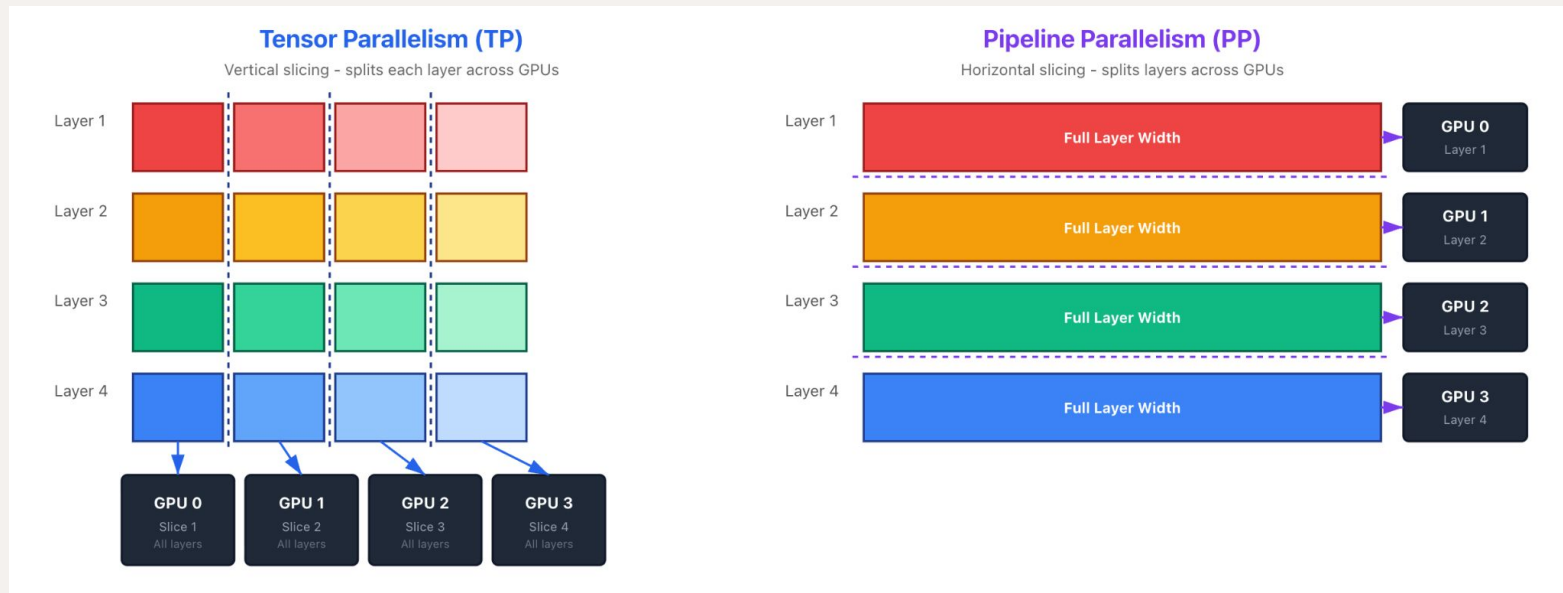
- **Small models (≤10B):** 1-2 T4, L4, A10G
- **Medium models (10B-70B):** 2-4 A10G/L40S or 1-2 A100 with TP
- **Large models (70B-500B):** Multiple A100/L40s/H100/H200 with TP or PP
- **Extreme models (500B+):** Multi-node H200/B200/B300 clusters with TP + PP
- **Note:**
 - ◆ [See the full list of GPU support](#) such as Nvidia, AMD, Intel, Google TPUs, etc.

GPU	Architecture	Memory (GB)	CUDA cores	Bandwidth (TB/s)	Interconnect	AWS instance (single-GPU examples)	Google Cloud instance (single-GPU examples)
Nvidia T4	Turing	16	2,560	0.32	PCIe 3	<code>g4dn.xlarge</code> (1 × T4)	<code>n1-standard-4</code> + 1 × T4
Nvidia L4	Ada	24	7,424	0.30	PCIe 4	<code>g6.xlarge</code> (1 × L4)	<code>g2-standard-4</code> (1 × L4)
Nvidia L40S	Ada	48	18,176	0.86	PCIe 4	<code>g6e.2xlarge</code> (1 × L40S)	N/A
Nvidia A10G	Ampere	24	9,216	0.60	PCIe 4	<code>g5.xlarge</code> (1 × A10G)	N/A
Nvidia A100-40G	Ampere	40	6,912	1.60	NVLink 3	<code>p4d.24xlarge</code> (8 × A100-40G)	<code>a2-highgpu-1g</code> (1 × A100-40G)
Nvidia A100-80G	Ampere	80	6,912	2.0	NVLink 3	<code>p4de.24xlarge</code> (8 × A100-80G)	<code>a2-ultragpu-1g</code> (1 × A100-80G)
Nvidia H100	Hopper	80	14,592	3.35	NVLink 4	<code>p5.48xlarge</code> (8 × H100)	<code>a3-highgpu-8g</code> (8 × H100)
Nvidia H200	Hopper HBM3e	141	16,896	4.8	NVLink 4	<code>p5e.24xlarge</code> (8 × H200)	<code>a3-ultragpu-8g</code> (8 × H200)
Nvidia B200	Blackwell	180	16,896	8.0	NVLink 5	<code>p6-b200.48xlarge</code> (8 × B200)	<code>a4-highgpu-8g</code> (8 × B200)

NVLink lets GPUs communicate faster, boosting throughput, lowering latency, and enabling large models across GPUs without memory or communication bottlenecks.

<https://docs.anyscale.com/llm/serving/gpu-guidance>

Tensor Parallelism (TP) & Pipeline Parallelism (PP)



Parallelism	What's split	Communication per layer	Best GPU Interconnect
TP	Matrix multiplies <i>inside</i> a layer	Multiple large collectives (<i>all-reduce/all-gather</i>)	Intra-node NVLink/NVSwitch
PP	Layers into stages	Activation send/rcv between neighboring stages	Prefer RDMA, RoCE, InfiniBand, EFA etc Inter-node Ethernet is OK, but can limit performance

Distributed LLM Serving Strategies (TP & PP)

	When to Use	Parallelism	Configuration Example
Single GPU	Model fits on a single GPU.	None	Run inference on that GPU.
Single-Node Multi-GPU	Model is too large for one GPU but fits on a single node with multiple GPUs.	<i>TP (with NVLink to reduce latency)</i>	tensor_parallel_size=4 (for a node with 4 GPUs).
Multi-Node Multi-GPU	Model is too large for a single node with multiple GPUs.	<i>TP + PP</i>	tensor_parallel_size=8 (GPUs per node), pipeline_parallel_size=2 (number of nodes).

Agent Assisted GPU Planning

The `/anyscale-workload-llm-serving` skill:

- Reads the HF model card + vLLM recipe for the model
- Estimates weights + KV fit for your target GPU, flags OOM before you deploy

The `/anyscale-platform-run` skill:

- Grounds compute fields against your cloud's real capacity (rejects instance types your cloud can't actually launch)

[/anyscale-workload-llm-serving](#) I would like to deploy the Qwen3.6 27B model with **4x L4 GPUs** for agentic coding. Is this a good fit? Please don't generate the configs; just help me do the calculation.

Memory budget on 4x L4

L4 = 24 GB each → 96 GB total. Per the sizing guidance for 24 GB GPUs, use [gpu_memory_utilization = 0.85](#):

usable VRAM = $96 \times 0.85 = 81.6$ GB

Weights (TP=4):

- BF16: $27B \times 2 \text{ bytes} = 54 \text{ GB} \rightarrow 13.5 \text{ GB/GPU}$
- FP8: $27B \times 1 \text{ byte} = \sim 27 \text{ GB} \rightarrow 6.75 \text{ GB/GPU}$ (L4 is Ada Lovelace – native FP8 support ✓)

KV cache per token – only the 16 full-attention layers count:

$2 \text{ (K+V)} \times 4 \text{ KV heads} \times 256 \text{ head_dim} \times 2 \text{ bytes (BF16)} = 4 \text{ KiB/token/layer}$
 $\times 16 \text{ layers} = 64 \text{ KiB per token}$

That's unusually cheap for a 27B – a conventional 64-layer transformer with the same GQA config would need 4x more. A **full 262K-token sequence costs 16 GB of KV cache**. The 48 DeltaNet layers add only a small constant recurrent state ($\sim 0.1\text{--}0.3$ GB per concurrent sequence).

What's left for KV cache (after weights, $\sim 1\text{--}1.5$ GB/GPU for activations + CUDA-graph capture buffers):

	BF16	FP8
Weights	54 GB	~ 27 GB
KV budget	~ 22 GB	~ 49 GB
Total KV capacity	$\sim 360K$ tokens	$\sim 800K$ tokens
Concurrent seqs @ 64K ctx	~ 5	~ 12
Concurrent seqs @ 128K ctx	$\sim 2\text{--}3$	~ 6
Concurrent seqs @ 262K ctx	1 (barely)	~ 3

Cross-checking with the standard sizing rule: FP8 gives $\text{min_gpus} = 27 / 20.4 = 1.3 \rightarrow 2$, and with the 2x safety factor → **4 GPUs, which is exactly your setup**. BF16 gives $54 / 20.4 = 2.65 \rightarrow 3$ minimum, $\sim 6\text{--}8$ with the safety factor – so 4x L4 in BF16 sits below the comfortable line, consistent with the tight KV numbers above.

For agentic coding specifically (the Step 1.5 check): you want 128K–256K of usable context. FP8 comfortably serves a few concurrent 128–262K agent sessions; BF16 realistically supports one long session at a time.

Performance caveats – the real limitation is speed, not fit

- **Decode throughput**. L4 has only ~ 300 GB/s memory bandwidth. Rough upper bound per token: $13.5 \text{ GB/GPU} \div 300 \text{ GB/s} \approx 45 \text{ ms} \rightarrow \sim 20 \text{ tok/s}$ in BF16, realistically 12–18 tok/s after PCIe all-reduce overhead (L4 nodes have no NVLink). FP8 roughly doubles that to $\sim 25\text{--}35 \text{ tok/s}$. For agentic coding with long generations, BF16 will feel sluggish; FP8 is the difference between ↓

LLM with Quantization

→ Reduces GPU memory constraints

- ◆ FP8 reduces the model size by half compared with BF16.
- ◆ NVFP4 reduces the model size to one-quarter of BF16*.

Note: some quantized LLMs labeled as NVFP4 but keep certain layers in BF16, FP8, or another higher-precision format.

→ Quality retention

- ◆ Well-calibrated FP8 and NVFP4 [retain over 99% of quality of original BF16 model](#)

→ GPU support of quantization varies

- ◆ Native FP8 (W8A8) needs Ada/Hopper/Blackwell
- ◆ NVFP4 needs Blackwell

/anyscale-workload-llm-serving I would like to deploy the Qwen3.6 27B model with RTX PRO 6000 GPU for agentic coding. Is this a good fit? Please don't generate the configs; just help me do the calculation.

Everything checks out – and one thing worth flagging up front: the name is exactly [Qwen/Qwen3.6-27B](#) (I was initially skeptical since 27B is an unusual Qwen size, but it's a real dense 27B released 2026-04-22). Here's the full sizing calc. No configs generated.

Fact	Value	Source
Model	Qwen3.6-27B – dense 27B, multimodal (VLM), reasoning (thinking-by-default)	HF card
Architecture	Hybrid: alternating Gated DeltaNet (linear/recurrent) + Gated Attention, 64 layers, hidden 5120, vocab 248,320	HF card
Attention (KV-bearing) heads	4 KV heads × 256 head-dim (GQA), Q=24	HF card
Native context	262,144 tokens (~1M extensible)	HF card
Released precision	BF16; official FP8 and NVFP4 checkpoints exist	HF / recipe
GPU	RTX PRO 6000 Blackwell = 96 GB GDDR7, 1.8 TB/s, native FP8 + NVFP4	NVIDIA/StorageReview
VLLM recipe for RTX Pro 6000	NVFP4 + --kv-cache-dtype fp8 + MTP spec-decode; needs vLLM ≥ 0.24.0	vLLM Recipes

Step 1 – Model weights
weights_GB ≈ params × bytes_per_param

Precision	bytes/param	27B weights	+ vision tower (~1-2 GB)
BF16	2.0	~54 GB	~56 GB
FP8	1.0	~27 GB	~29 GB
NVFP4 (4-bit)	~0.5 (+scales; embeds/norms kept higher)	~15-18 GB	~17-20 GB

Note: the 248K vocab × 5120 embedding matrix (~1.3B params, often kept in higher precision) is why NVFP4 lands closer to ~16 GB than the naive 13.5 GB.

Step 2 – KV cache per token

Only the **Gated Attention** layers grow with sequence length; the DeltaNet layers hold a *fixed-size*

$KV_per_token = 2(K,V) \times num_kv_heads(4) \times head_dim(256) \times n_attn_layers \times bytes$
Per attention layer per token = $2 \times 4 \times 256 = 2,048$ elements.

The exact count of full-attention layers isn't in the card (the pattern is 1:1 alternating → ~32 choice):

n_attn_layers	KV/token (fp8)	Full 262K seq	128K seq	32K seq
16 (if 3:1 ratio)	32 KiB	8 GiB	4 GiB	1 GiB
32 (assumed 1:1)	64 KiB	16 GiB	8 GiB	2 GiB
64 (pessimistic)	128 KiB	32 GiB	16 GiB	4 GiB

(bf16 KV cache = 2× these.)

Step 3 – Fit on one 96 GB card (NVFP4, fp8 KV – the recipe path)

Total VRAM 96 GB
× gpu_memory_utilization 0.9 → ~86 GB usable
– NVFP4 weights + vision ~18 GB
– framework + CUDA graphs ~4-5 GB
– MTP speculative heads + spec-KV ~1-2 GB

KV-cache budget ~60-62 GB

At 64 KiB/token → 62 GB / 65,536 B ≈ 0.95M tokens of aggregate KV budget.

Translated to concurrency for agentic coding:

Working context / request	Concurrent requests
32K	~29
128K	~7
Full 262K	~3.6

Verdict: yes – strong fit, single GPU, no tensor parallelism

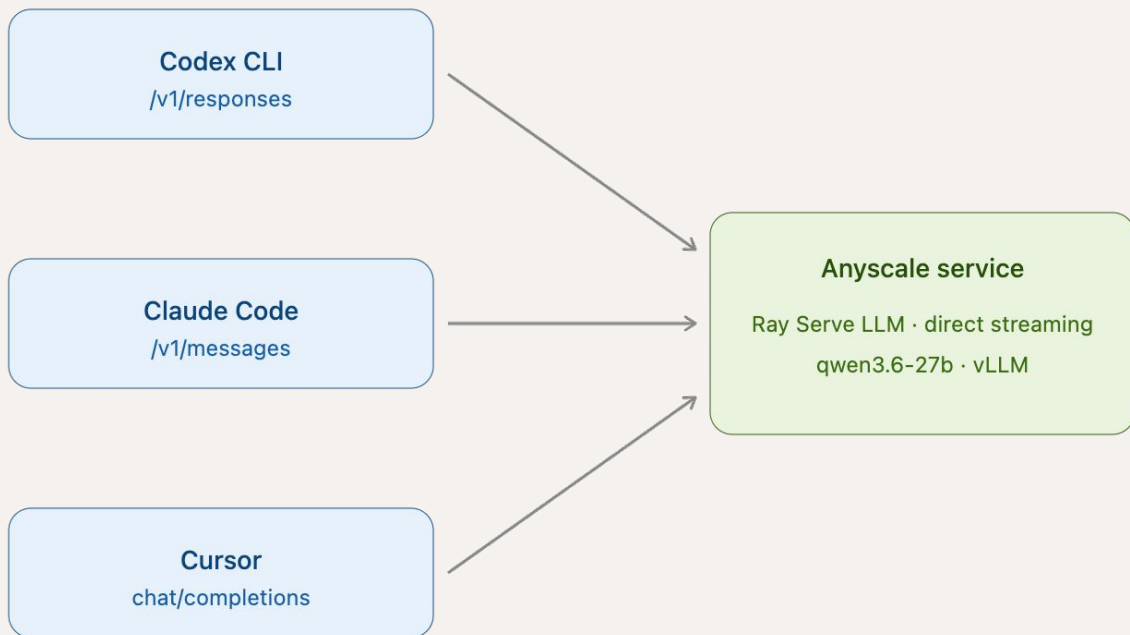
Step 3: Set up Serving Deployment Configurations

vLLM Engine Parameters

```
engine_kwargs=dict(
    tensor_parallel_size=4,          # 4x L4 GPUs on one g6.12xlarge (PCIe, no NVLink)
    max_model_len=131072,           # 128K
    gpu_memory_utilization=0.85,
    # kv_cache_dtype left at the vLLM default (bf16) – no tuning here; see Part 3.
    # Validated on this shape: 652,346-token GPU KV cache, 10.38 GiB/GPU,
    # 4.98x raw concurrency at 128K (vLLM 0.22.0 engine log).
    max_num_seqs=16,
    max_num_batched_tokens=8192,
    enable_prefix_caching=True,
    trust_remote_code=True,
    reasoning_parser="qwen3",
    tool_call_parser="qwen3_coder",
    enable_auto_tool_choice=True,
    # Image input ENABLED by default (Qwen3.6-27B is a VLM). Set image:0 for a text-only endpoint.
    # NOTE: verified on the Part 3 single-GPU 96GB shape; this 4x L4 (24GB) baseline is tighter and
    # was not separately load-tested – if you hit OOM at startup, lower max_pixels (mm_processor_kwargs)
    # or reduce max_model_len.
    limit_mm_per_prompt={"image": 4, "video": 0},
),
```

[Agent](#) researches the **HF model card + vLLM recipe + official Qwen docs** for the model, setting the right parameters in vLLM.

Step 4: Deploy on Anyscale & Connect with your agents



Need to enable the [Direct Streaming in Ray Serve LLM](#)

Setup Steps & Known Search Issues [[link](#)]

- **Cursor Setup:**

`WebSearch` and `WebFetch` work as expected.

- **Claude Code Setup:**

`WebSearch` and `WebFetch` are Anthropic server-side tools and are not available for Claude Code. Use a local search MCP instead.

- **Codex Setup:**

A local search MCP is also required. However, [MCP tool invocation has regressed for custom and local providers](#). As a workaround, use Codex version `0.116.0` or earlier.



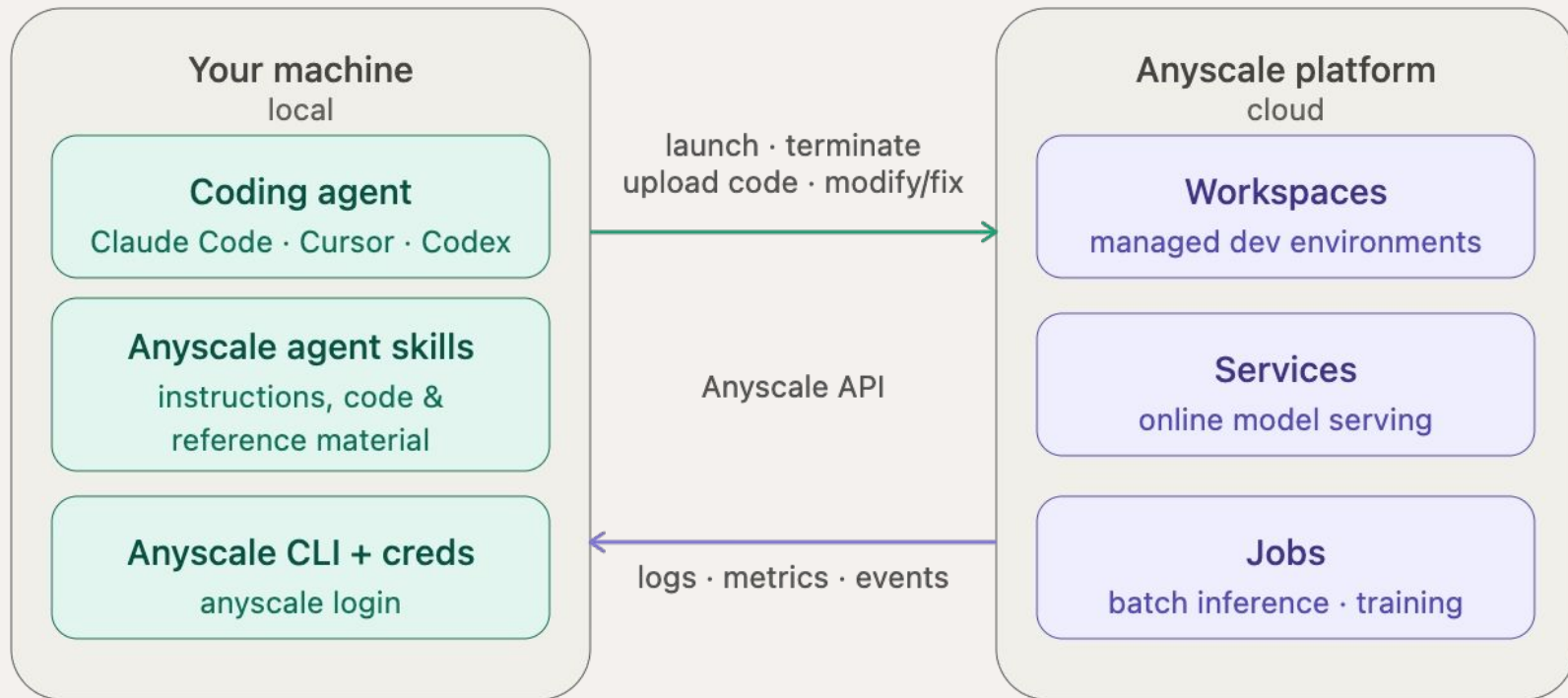
LLM Serving for Coding Agent Hands on Lab

Important Notes:

- Due to the AWS load balancer quota and GPU limit in the Anyscale Cloud, we are running the LLM in an Anyscale Workspace with 4xL4 GPU today, which is NOT ideal.
- **For production, deploy the LLM as an Anyscale Service** and follow the instructions in this Github repo:

<https://github.com/anyscale/llm-serving-for-coding-agents/tree/main>

How Agent Skills Work



Install Anyscale Agent Skills Locally in your Laptop

```
pip install -U anyscale
```

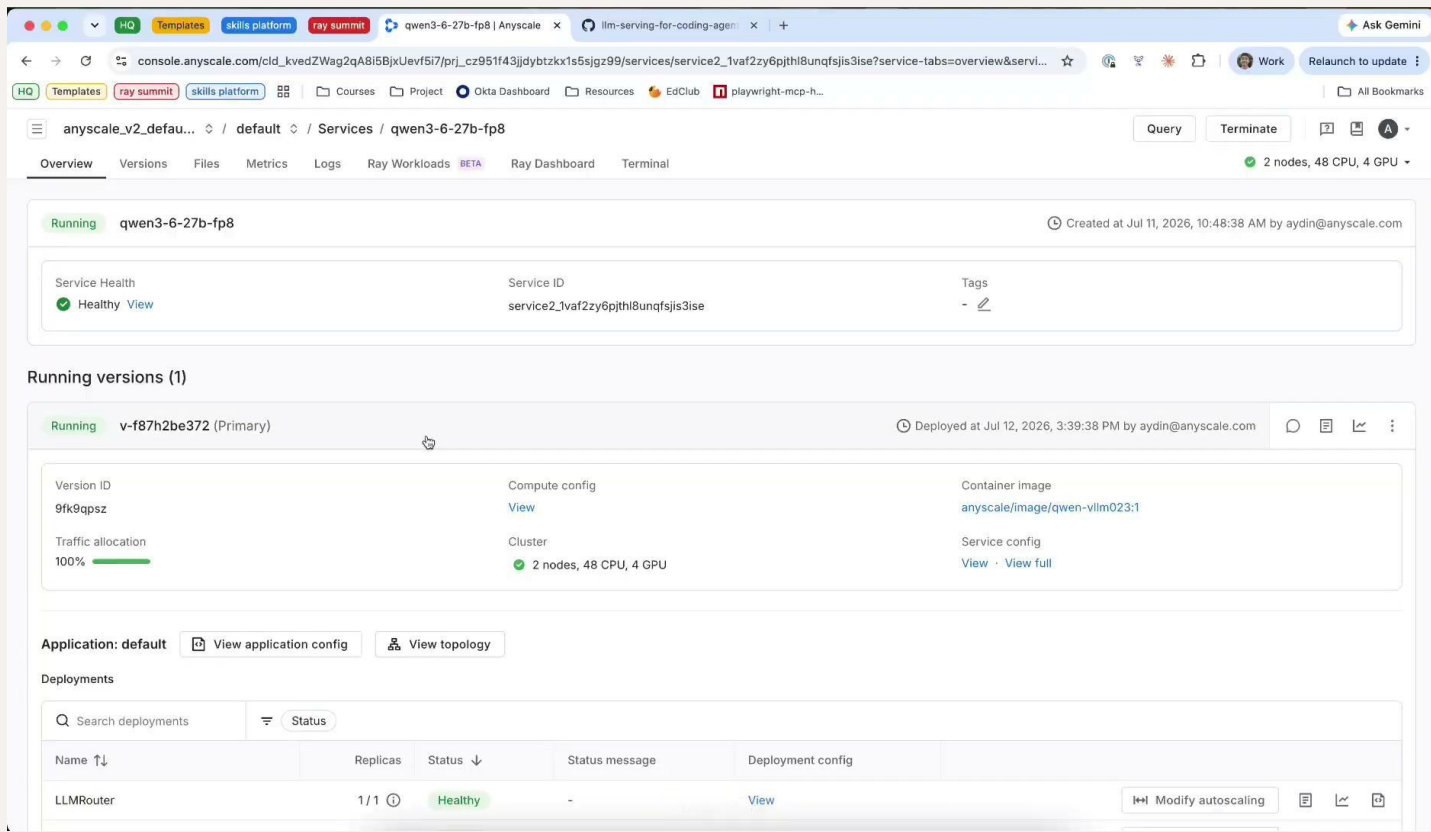
```
anyscale login
```

```
anyscale skills install
```

Prompt - Hands on Lab

[/anyscale-workload-llm-serving](#) Deploy qwen3.6-27b with tool calling enabled on 4xL4 with ray 2.57.0 and vllm 0.25, weights located at s3://llm-guide-use2/data/ray-serve-llm/hf_repo/Qwen3.6-27B-FP8/. No optimization needed. Use model_id="qwen3.6-27b". Deploy in the workspace llm-post-training-serving with serve run.

Reference: Cursor Integration with LLM Service



The screenshot displays the Anyscale console interface for a service named 'qwen3-6-27b-fp8'. The top navigation bar includes tabs for 'HQ', 'Templates', 'skills platform', 'ray summit', and 'qwen3-6-27b-fp8 | Anyscale'. The main content area shows the service's overview, including its health status (Healthy), service ID, and tags. Below this, the 'Running versions (1)' section details the primary version 'v-f87h2be372', including its version ID, compute configuration, container image, traffic allocation (100%), and cluster details (2 nodes, 48 CPU, 4 GPU). The 'Application: default' section provides links to view the application config and topology. The 'Deployments' section at the bottom shows a table with columns for Name, Replicas, Status, Status message, and Deployment config, listing the 'LLMRouter' deployment with 1/1 replicas in a 'Healthy' state.

Service Overview:

- Service Name:** qwen3-6-27b-fp8
- Service Health:** Healthy
- Service ID:** service2_1vaf2zy6pjl8unqfsjs3ise
- Tags:** -
- Created at:** Jul 11, 2026, 10:48:38 AM by aydin@anyscale.com

Running versions (1)

- Version:** v-f87h2be372 (Primary)
- Version ID:** 9fk9qpsz
- Compute config:** [View](#)
- Container image:** [anyscale/image/qwen-vllm023:1](#)
- Traffic allocation:** 100%
- Cluster:** 2 nodes, 48 CPU, 4 GPU
- Service config:** [View](#) · [View full](#)
- Deployed at:** Jul 12, 2026, 3:39:38 PM by aydin@anyscale.com

Application: default [View application config](#) [View topology](#)

Deployments

Name ↑↓	Replicas	Status ↓	Status message	Deployment config
LLMRouter	1 / 1 ⓘ	Healthy	-	View

[Modify autoscaling](#) [View](#) [View full](#)

LLM Serving Optimizations for Coding Agents

Characteristics of Coding Agents

- **Prefill-heavy:** huge prompts (context / memory), relatively short completions
- **High prefix KV cache reuse:** same system prompt, agent skills, MCPs, repo context, and history re-sent across agent steps
- **Low latency SLOs:** low inter-token latency for streaming tool calling and edits
- **Burst and peak concurrency:** Usage can spike across teams or organizations, ranging from dozens to thousands of concurrent developers running agents.

Benchmarking with the Real Claude Code Sessions

- **Synthetic prompts $\neq$ real coding-agent traffic.** Common benchmarking tools and data missing prompts with agent loops, tool calls, and prefix-cache reuse for multi-turns. Input prompt is $\sim 70K$ tokens in average for coding agents.
- **Claude Code already logs the real thing.** Each session is a JSONL transcript at `~/.claude/projects/<project>/<session-id>.jsonl`.
- **Each record carries the full payload + timestamps.** The complete prompt, system prompt, agent skills, MCP tool calls, and per-request token usage — recovering real sizes and cache-hit ratios.
- **Replay real sessions $\rightarrow$ drives the optimization work.** A timestamped event stream we replay against the model instead of synthetic prompts.

Optimization Results for RTX Pro 6000

Each row compares one knob off vs on, on the same hardware.

#	Knob	Off	On	Result	Default
0	ENABLE_NVFP4_WEIGHTS	FP8, ~27 GB	NVFP4, ~22 GB	~5 GB smaller (~19%)	On
1	ENABLE_FAST_MODEL_LOADING	HF download, ~85 s	RunAI Streamer, ~25 s	3.4× faster load	Off
2	ENABLE_COMPILE_CACHE	Recompile, 74.5 s	Prebuilt cache, 8.8 s	8.5× faster compile	On only without MTP
3	ENABLE_FP8_KV_CACHE	bf16 KV, ~3.3× concurrency at 256K	fp8 KV, full 256K	6.53× concurrency	On
4	ENABLE_CUDA_GRAPHS	Eager, 15.9 tok/s	Graphs, 45.6 tok/s	2.87× decode (FP8 measurement)	On
5	ENABLE_SPEC_DECODE	NVFP4 base, 65 tok/s	NVFP4 + MTP, 121 tok/s	1.86× single-stream decode	On

NVFP4 KV crashes due to kernel constraints on this GPU

Spec decode is on by default because the tutorial targets coding-agent traffic, where lower TPOT during multi-token generation matters more than shaving about a minute from cold weight load. Fast model loading is kept as an opt-in because RunAI Streamer and MTP cannot currently coexist (validated through vLLM 0.23.0; [vllm#42060](#)). Prefix routing is off by default because the single-user replay data in this tutorial does not need replica affinity; see [Prefix Routing](#) for when to opt in. The built-in router also needs the ray-llm 2.57 direct-streaming fix ([ray#64328](#)). See [INCOMPATIBILITIES.md](#) for combinations that cannot coexist. The MTP graph does not use the prebuilt no-MTP compile cache, so enabling MTP automatically disables cache restoration.

Optimization Walkthrough

<https://github.com/anyscale/llm-serving-for-coding-agents/tree/main/part3-optimize>

Cost Estimation & Comparison

GPU: RTX Pro 6000

Input	Planning value
On-demand GPU price	~\$4.00/hr
Active sessions per GPU	~24
Developer duty cycle	~25%
Planning registered developers per GPU	~50
Input tokens per turn	~70K

<https://github.com/anyscale/llm-serving-for-coding-agents/blob/main/part3-optimize/notes/COST-ESTIMATE.md>

Commercial pricing comes in two scenarios, and the comparison should be made against both. They are largely the **same usage billed two ways**; what flips an org from the first to the second is the vendor's seat cliff, not a jump in how much developers actually use the tool:

- Scenario 1 - Subscription seats, ~\$200/dev-month (under ~150 seats).** A Claude Max 20x or Cursor Ultra seat is a flat monthly price that *includes* usage, so it caps a heavy user at about \$200/month. Past the seat's limit they are throttled, not billed more.
- Scenario 2 - Token-metered billing, ~\$800/dev-month (past the ~150-seat cliff).** Per the [Pylon seat-cliff post](#), crossing about 150 Anthropic seats forces the Enterprise tier: seats stop including usage and every token bills at standard API rates. Usage did not change, but the same active engineer's real consumption now bills in full, about \$800/dev-month. Pylon's bill jumped ~3.5x overnight; they measured about \$780/dev.

So the ~4x gap is not heavier usage. Under the cliff, seats *bundle and cap* usage at ~\$200; past it, per-token billing *exposes* the full cost of the same engineer.

Self-hosted mode	GPU cost	Planning cost at 50 devs/GPU	Savings vs \$200 seats	Savings vs \$800 token bill
Always-on, on-demand	~\$2,900/mo	~\$58/dev-mo	~71%	~93%
Work-hours, on-demand	~\$840/mo	~\$17/dev-mo	~92%	~98%

Break-even is small because the GPU is a shared fixed cost:

Self-hosted mode	Break-even vs \$200 seats	Break-even vs \$800 token bill
Always-on, on-demand	~15 devs	~4 devs
Work-hours, on-demand	~4 devs	~1 dev

Note: \$800 / month is estimated from [1.4 million / year for 150 engineers](#)

To Deploy the optimized service yourself

Enter this prompt with Claude Code with Agent skills:

[/anyscale-workload-llm-serving](#) Deploy the optimized LLM service follow the guidance in this repo:
<https://github.com/anyscale/llm-serving-for-coding-agents/tree/main/part3-optimize>

The agent will use this code as a template and adapt it as needed:

- Tip #1: You can suggest the GPU type
- Tip #2: You can also try deploying other LLMs

Hybrid Routing and Practical Considerations

Hybrid LLM Routing Strategy

Scenario 1 — Route by task complexity

- Simple tasks and agent skills run on small open-source LLMs to save cost
- Critical or complex tasks stay on frontier models (Opus / Fable / GPT)

Scenario 2 — OSS at capacity → fail over to Claude / Codex close source LLMs

- When open-source LLMs face spiking input volume or can't autoscale fast enough, fall back to a closed-source subscription (default Claude / Codex)

Scenario 3 — Closed-source out of credit → fail over to OSS

- When Claude Code runs out of credits, fall back to open-source LLMs to keep work flowing

Routing Between hosted LLMs and Claude Models via LiteLLM

Unified — both [LiteLLM](#) gateway + Ray Serve LLM are deployed together as Anyscale Services

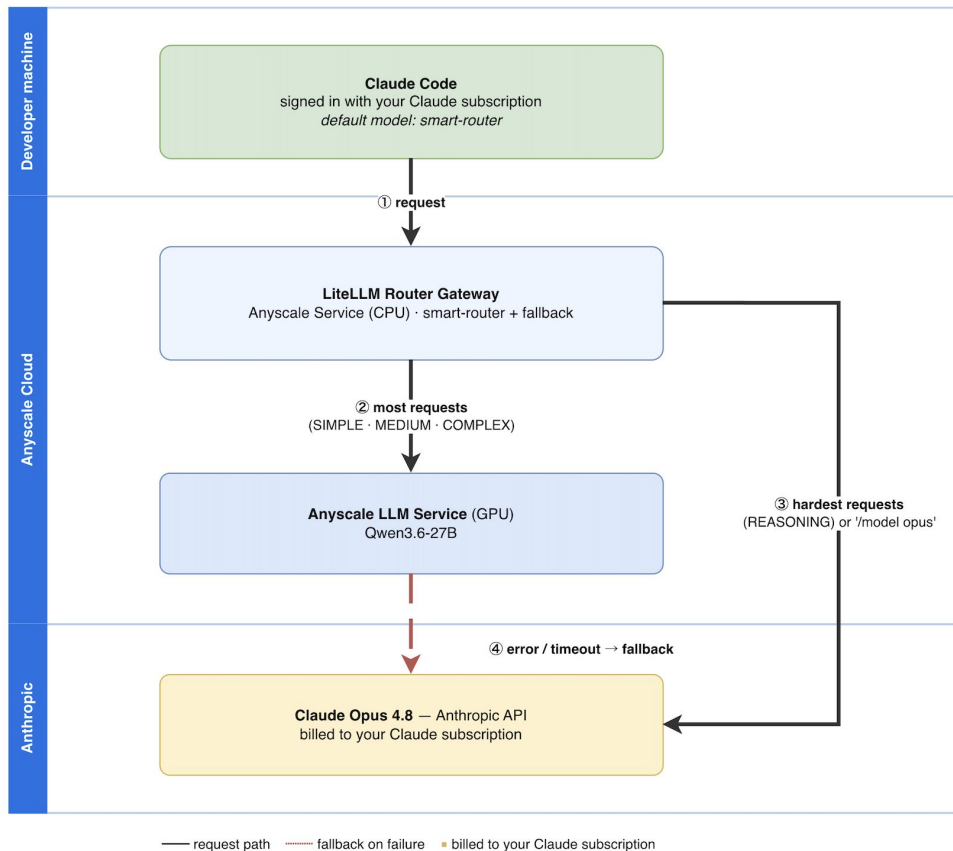
Resilient — Claude subscription auto-fallback when GPUs are scarce or too many requests for open LLM service

Adaptive — prompt-based routing, route based on keywords or skills names

Extensible — easily add more Claude models (Table 5) or route multiple open LLMs hosted on Ray Serve LLM

Traceable — collect agent traces through [Langfuse](#) integration, turning real traffic into curated fine-tuning data for the open LLMs

LiteLLM Router Gateway for Claude Code

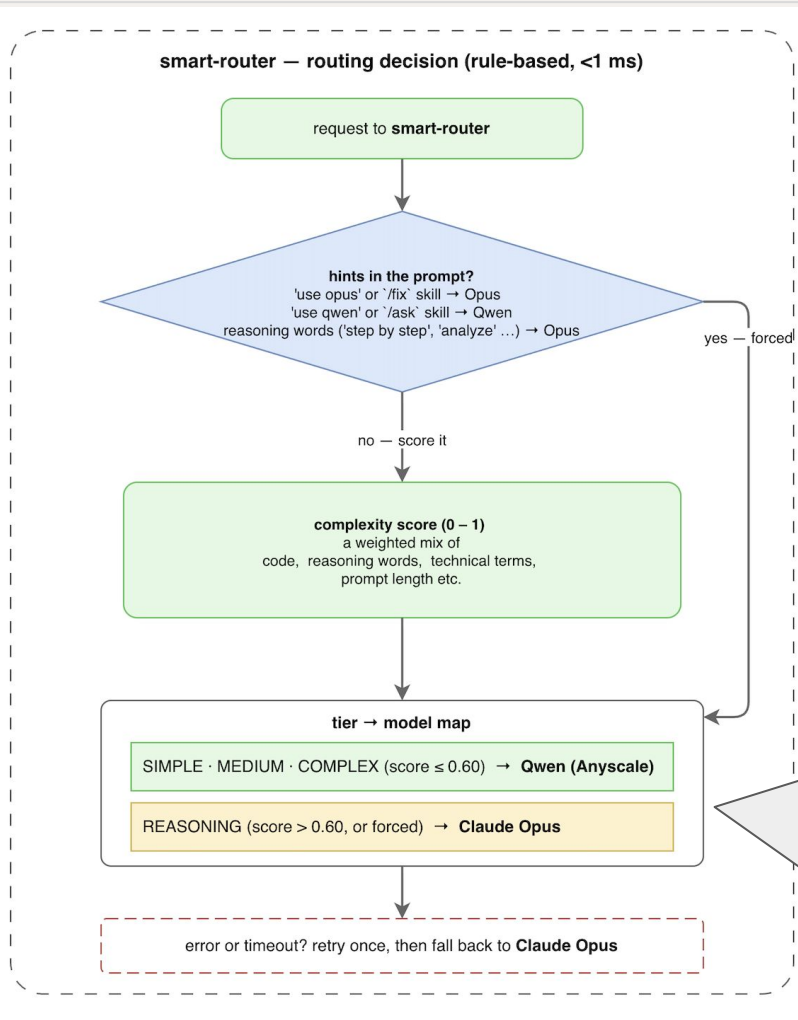


Routing Decision

Fast and configurable — keyword matching in under 1 ms, with configed keywords, thresholds, and weights

Keywords & skill-aware routing — define keywords or skills names that map specific models to specific tasks

Beyond keywords — LiteLLM also supports LLM classifiers plus cost-, latency-, and adaptive-routing options (see [auto-routing docs](#))



Note: The tiers are fixed, ordered values

- SIMPLE
- MEDIUM
- COMPLEX
- REASONING

Deploy the Router Service

Enter this prompt with Claude Code with Agent skills:

`/anyscale-platform-run` Deploy the optimized LiteLLM router service on Anyscale and set up a local terminal command (**claude-code-with-router**) to launch the Claude Code with the router service follow the guidance in this repo:

<https://github.com/anyscale/llm-serving-for-coding-agents/tree/main/part4-litellm-router>

Enterprise Roll out and Admin Configs

- Admin only pushes two JSON files via Anthropic admin console

```
{
  "mcpServers": {
    "brave-search": {
      "command": "npx",
      "args": [ "-y", "@brave/brave-search-mcp-server", "--transport", "stdio"],
      "env": { "BRAVE_API_KEY": "YOUR-BRAVE-API-KEY" }
    }
  }
}
```

[managed-mcp.json](#)

- Developer launch Claude code as usual

- See more details in:
 - <https://github.com/anyscale/llm-serving-for-coding-agents/tree/main/part5-enterprise-rollout>

```
{
  "$schema": "https://json.schemastore.org/claude-code-settings.json",
  "env": {
    "ANTHROPIC_BASE_URL": "https://YOUR-GATEWAY-HOST.s.anyscaleuserdata.com",
    "ANTHROPIC_CUSTOM_HEADERS": "x-litellm-api-key: Bearer sk-paste-the-gateway-master-key",

    "ANTHROPIC_MODEL": "smart-router",
    "ANTHROPIC_DEFAULT_OPUS_MODEL": "smart-router",
    "ANTHROPIC_DEFAULT_SONNET_MODEL": "smart-router",
    "ANTHROPIC_DEFAULT_HAIKU_MODEL": "smart-router",
    "ANTHROPIC_SMALL_FAST_MODEL": "smart-router",

    "API_TIMEOUT_MS": "1200000"
  }
}
```

[managed-settings.json](#)

Resources - LLM Serving & Optimizations

Today's code repo [<https://github.com/anyscale/-llm-serving-for-coding-agents>]

LLM Agent Skills [<https://docs.anyscale.com/agent-skills>]

LLM serving examples & template [<https://console.anyscale.com/template-preview/deployment-serve-llm>]

Anyscale LLM Serving Docs [<https://docs.anyscale.com/llm/serving>]

Ray Serve LLM Docs [<https://docs.ray.io/en/master/serve/llm/index.html>]

GPU Guidance for LLM serving [<https://docs.anyscale.com/llm/serving/gpu-guidance>]

Tune parameters for LLMs on Anyscale services [<https://docs.anyscale.com/llm/serving/parameter-tuning>]

Optimize performance for Ray Serve LLM [<https://docs.anyscale.com/llm/serving/performance-optimization>]

Ray Serve Autoscaling for LLMs [<https://docs.anyscale.com/llm/serving/parameter-tuning#ray-serve-autoscaling-configuration>]

Understand LLM latency and throughput metrics [<https://docs.anyscale.com/llm/serving/benchmarking/metrics>]

Q&A

15 mins Break

(Please stop your Ray Serve LLM in the workspace with `serve shutdown`)

Post-Training and RL with Ray

Agenda

- Demo Launch
- Intro to Post-Training
- Intro to RLVR
- Case Study: Post-Training a Terminal Agent
 - Building Evals + Envs
 - RL Systems
- Demo Walkthrough: Fully Async RL with SkyRL
 - Observability for RL on SkyRL

Demo Kickoff

/anyscale-workload-llm-post-training can you git clone <https://github.com/NovaSky-AI/SkyRL>, and run the script `examples/train/fully_async/fully_async_run_gsm8k.sh` in it on the workspace <WORKSPACE NAME HERE> but using train batch size 64 and running training for just 1 epoch?

First, download the qwen2.5 model checkpoint from the s3 path `s3://llm-guide-use2/qwen2.5-1.5b/` to `/mnt/cluster_storage`. Then download the data files from `s3://llm-guide-use2/gsm8k/` to `/mnt/cluster_storage` as well.

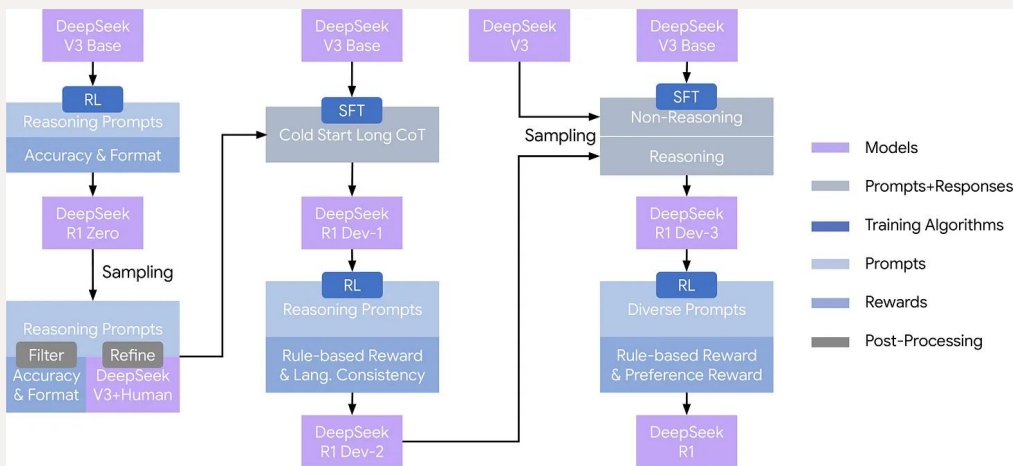
Update the script to use the new paths for model and dataset at `/mnt/cluster_storage` and then use the console as the logger instead of wandb if WandB is not available.

Intro to Post-Training

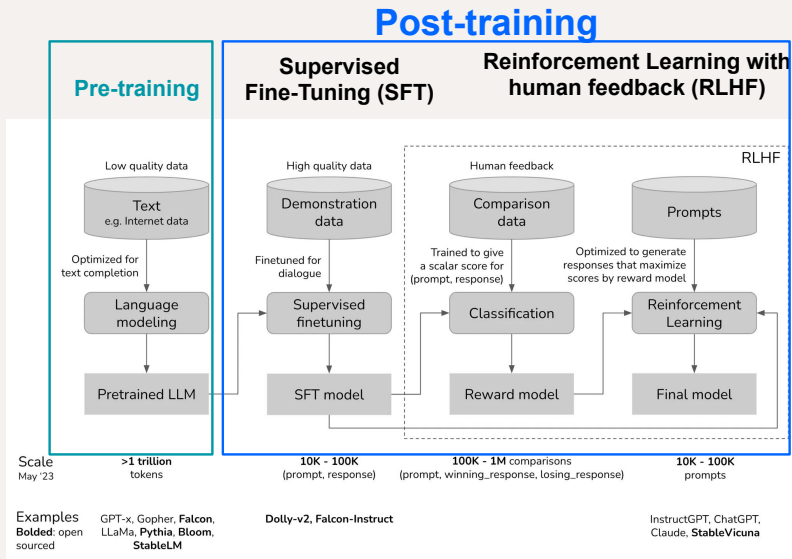
What is Post-Training? 2023

- Everything that comes after pre-training!
- SFT, RLHF (DPO, KTO), RLVR, OPD, MOPD, OPSD,

2025

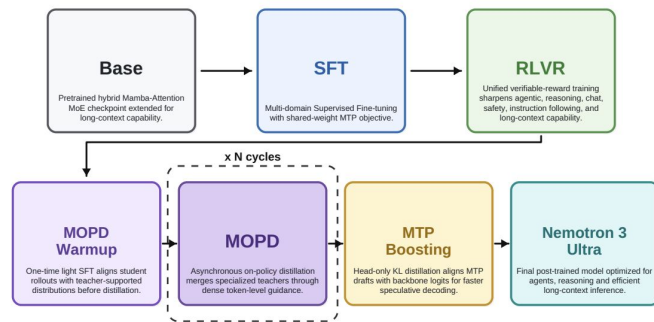


<https://rlhfbok.com/course#lectures>



2026

Figure 9 | Overview of the post-training pipeline for NemoTron 3 Ultra.



What does that mean today?

- Every model you interact with has gone through extensive post-training already
- What post-training should you add?
 - **Continual Pretraining (CPT)**: to adapt the model to a new domain
 - Given a large existing text corpus
 - **Supervised Fine Tuning (SFT)**: to teach the model to respond to prompts/questions
 - Given existing prompt/response pairs
 - **RL from Human Feedback (RLHF)**: to teach the model subjective human preferences
 - Given human preference annotations
 - **RL from Verifiable Rewards (RLVR)**: to teach the model how to solve complex problems that require reasoning or tools
 - Given verifiable tasks

What does that mean today?

- Every model you interact with has gone through extensive post-training already
- What post-training should you add?
 - **Continual Pretraining (CPT)**: to adapt the model to a new domain
 - Given a large existing text corpus
 - **Supervised Fine Tuning (SFT)**: to teach the model to respond to prompts/questions
 - Given existing prompt/response pairs
 - **RL from Human Feedback (RLHF)**: to teach the model subjective human preferences
 - Given human preference annotations
 - **RL from Verifiable Rewards (RLVR)**: to teach the model how to solve complex problems that require reasoning or tools
 - Given verifiable tasks

Why LLM Post-training

- Frontier models are expensive, and the gap with OSS models has never been lower

Show Filters

Models Labs

Rank	Model	Net Improvement	Confirmed Success	Praise vs Complaint	Steerability
1 1 ~ 7	A Claude Fable 5 (High) Anthropic · Proprietary	▲ 12.58% ±2.19%	▲ 10.26% ±4.23%	▲ 24.42% ±7.82%	▲ 12.93% ±4.49%
2 1 ~ 9	A Claude Opus 5 (Max) Anthropic · Proprietary	▲ 11.88% ±2.81%	▲ 17.56% ±4.69%	▲ 25.63% ±9.98%	▲ 1.81% ±7.26%
3 1 ~ 7	A Claude Opus 5 (High) Anthropic · Proprietary	▲ 11.73% ±1.66%	▲ 16.60% ±3.11%	▲ 24.40% ±6.28%	▲ 5.49% ±3.76%
4 1 ~ 11	G GPT 5.6 Sol (xHigh) OpenAI · Proprietary	▲ 10.02% ±1.63%	▲ 8.61% ±3.32%	▲ 21.90% ±6.10%	▲ 8.96% ±3.09%
5 1 ~ 10	K Kimi K3 (Max) Moonshot · Kimi K3 license	▲ 9.91% ±1.19%	▲ 14.23% ±7.40%	▲ 17.45% ±4.20%	▲ 9.67% ±2.18%
6 1 ~ 12	A Claude Opus 4.8 (Thinking) Anthropic · Proprietary	▲ 9.43% ±1.64%	▲ 8.75% ±2.75%	▲ 21.05% ±5.25%	▲ 9.77% ±2.74%
7 1 ~ 14	A Claude Sonnet 5 (High) Anthropic · Proprietary	▲ 8.57% ±2.00%	▲ 7.95% ±3.98%	▲ 16.12% ±7.51%	▲ 6.74% ±4.10%
8 3 ~ 13	G GPT 5.5 (xHigh) OpenAI · Proprietary	▲ 8.49% ±0.95%	▲ 5.82% ±1.94%	▲ 12.91% ±3.38%	▲ 8.24% ±1.73%
9 3 ~ 13	A Claude Opus 4.7 (Thinking) Anthropic · Proprietary	▲ 8.19% ±1.26%	▲ 6.19% ±2.65%	▲ 11.20% ±4.39%	▲ 9.65% ±2.47%
10 4 ~ 13	G GPT 5.5 (High) OpenAI · Proprietary	▲ 7.89% ±0.88%	▲ 5.63% ±1.76%	▲ 11.31% ±3.13%	▲ 9.18% ±1.53%
11 5 ~ 15	A Claude Opus 4.7 Anthropic · Proprietary	▲ 7.37% ±1.31%	▲ 4.71% ±2.72%	▲ 12.28% ±4.45%	▲ 8.87% ±2.47%
12 6 ~ 15	Z GLM 5.2 (Max) Z.ai · MIT · SiliconFlow	▲ 7.08% ±0.96%	▲ 9.08% ±1.94%	▲ 13.79% ±3.42%	▲ 5.96% ±1.69%

Enterprises are increasingly choosing to post-train OSS models in 2026

Composer: Building a fast frontier model with RL

Oct 29, 2025 by Cursor Team

Composer is our new agent model designed for software engineering intelligence and speed. On our benchmarks, the model achieves frontier coding results with generation speed four times faster than similar models.

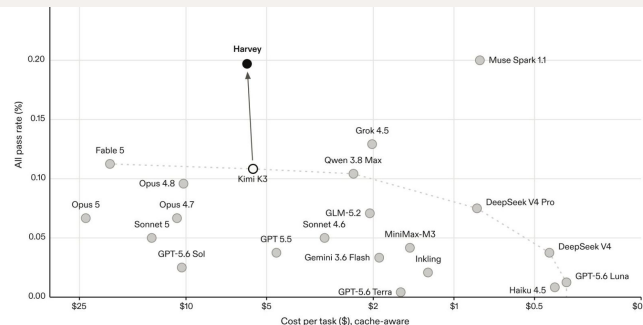
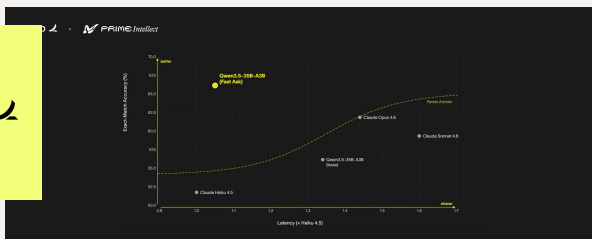


<https://cursor.com/blog/composer>

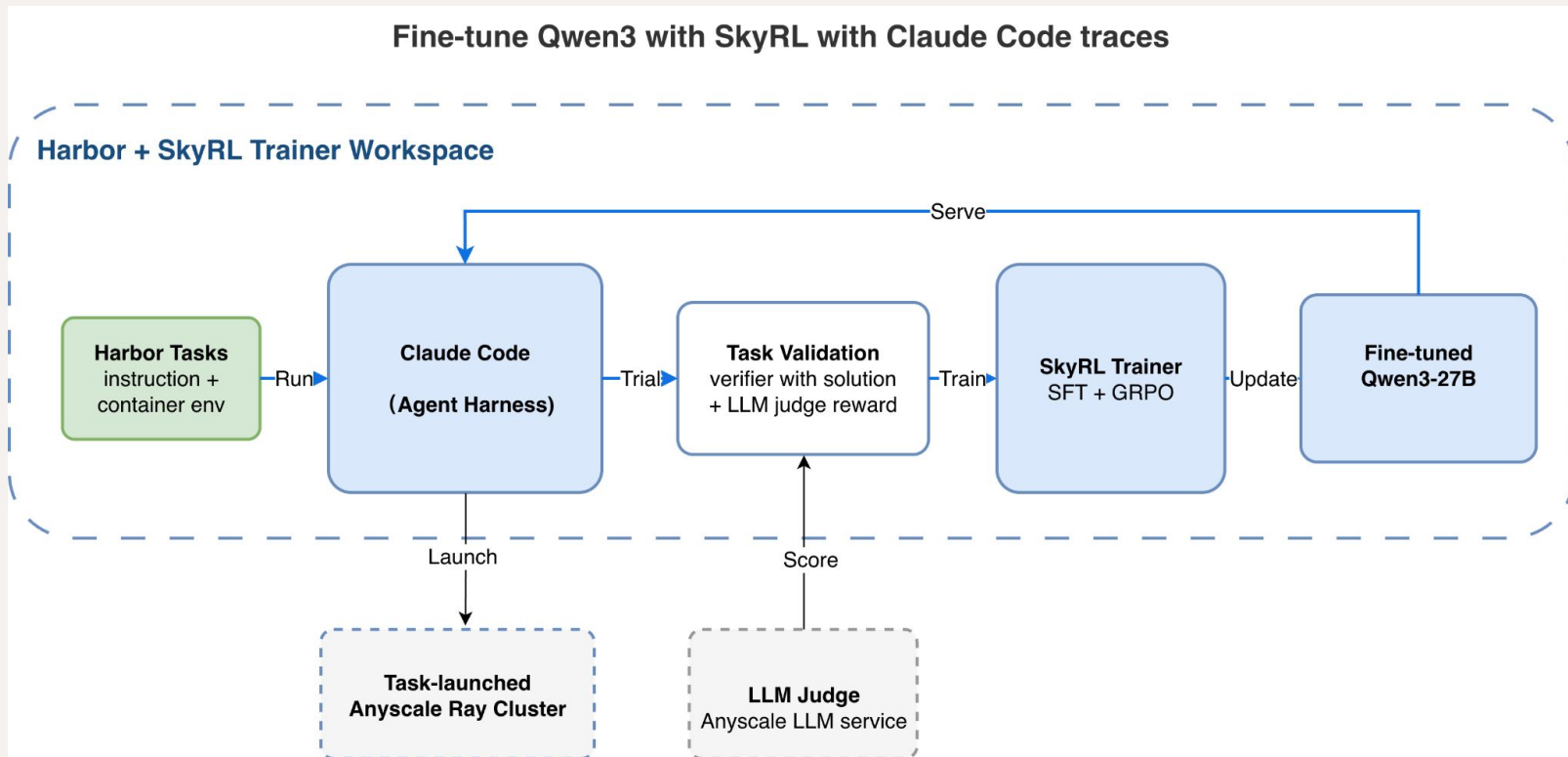
SWE-1.7: Frontier Intelligence at a Fraction of the Cost



ramp

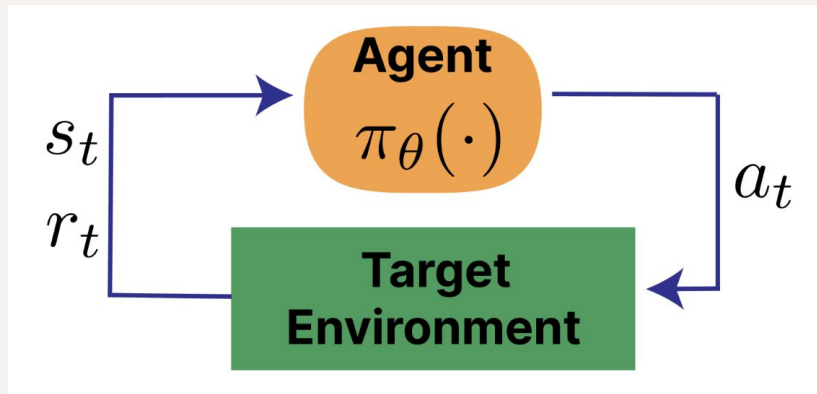


Goal: Continual Learning



Intro to RLVR

What is RL?



- Learning by trial and error
- An **agent** takes actions in an **environment**
 - The agent acts according to its **policy**
 - The sequence of actions is a **rollout/trajectory**
 - We assign a **reward** to the outcome which is used to update the policy

What is GRPO (Group Relative Policy Optimization)?

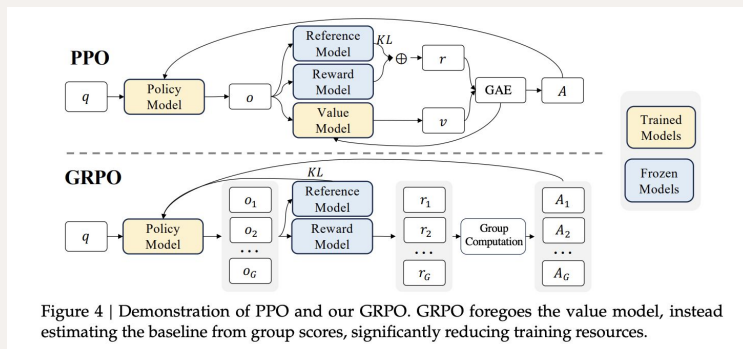
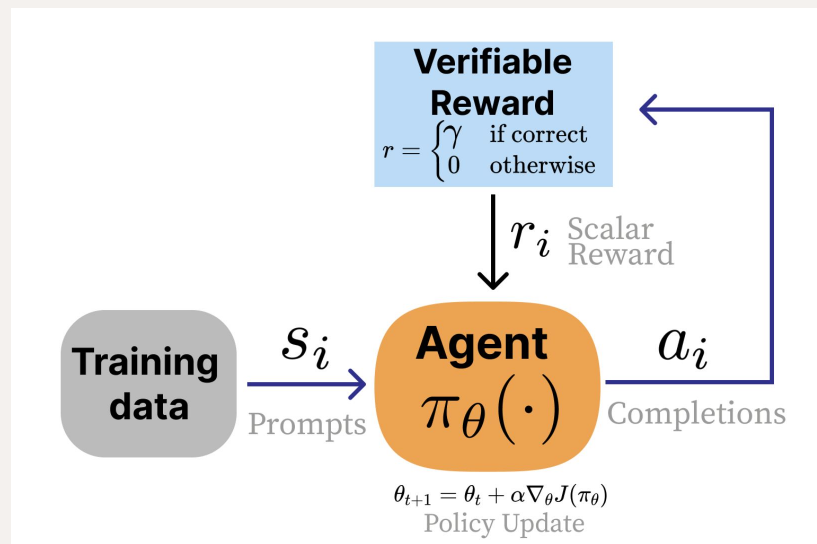


Figure 4 | Demonstration of PPO and our GRPO. GRPO foregoes the value model, instead estimating the baseline from group scores, significantly reducing training resources.

- Approximation of the PPO (Proximal Policy Optimization) algorithm - in GRPO, for a given prompt we:
 - Generate N rollouts with temperature > 0
 - Compute rewards for each rollout
 - Compute how well each rollout does relative to rest of group (advantage)
 - Train our model using the PPO loss
 - Repeat!

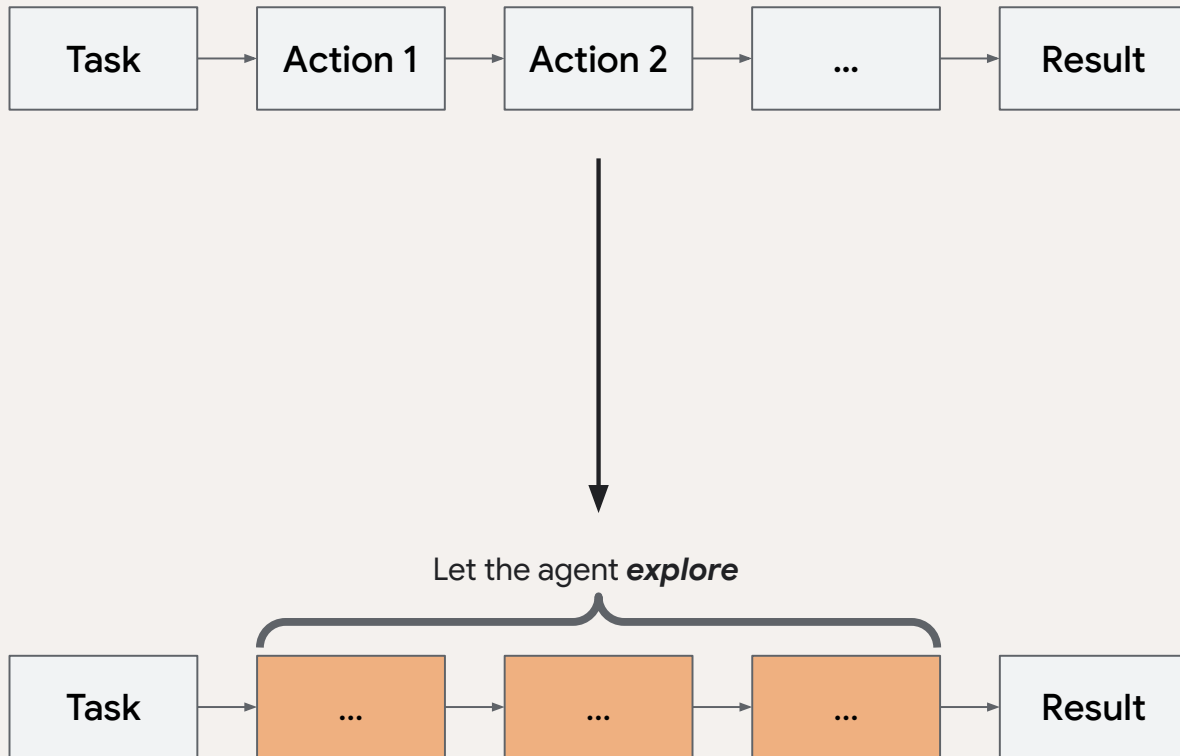
What is RLVR?

- Reinforcement Learning from **Verifiable Rewards**
 - Don't need a reward model or human annotations - final answers can be directly verified
 - Examples
 - Math - is the answer right?
 - Coding - do test cases pass?
 - Often also uses **rubrics**
 - For scaling RL to domains without clearly verifiable rewards



Why RL vs SFT for LLMs?

- SFT (and other offline methods) require manually capturing "agentic traces" for multi-hour, autonomous workflows
- RL allows you to specify the *what* (expected outcome) rather than the *how* (exact sequence)



Case Study: RL for a Terminal Agent

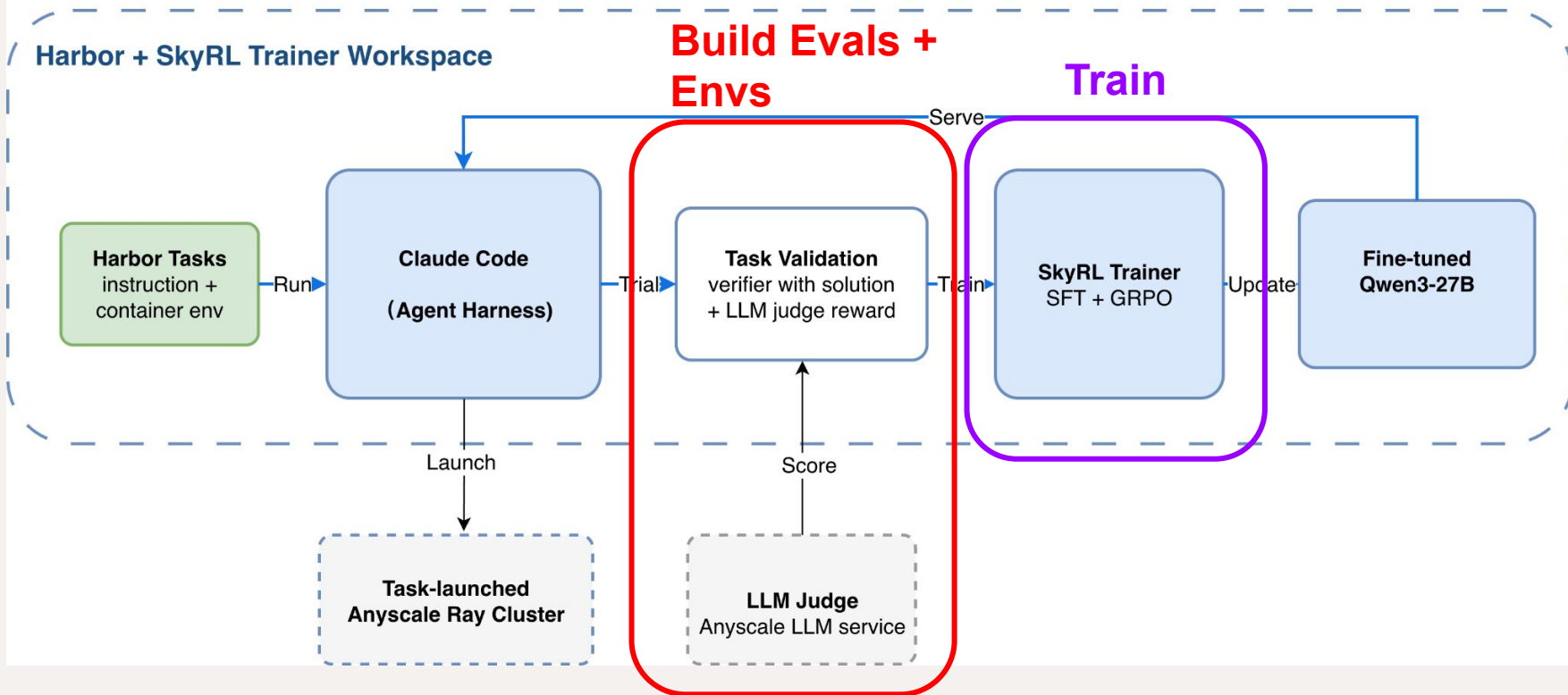
RL Playbook

- **Goal:** Training with multi-turn (20+), long context (64K+) agentic tasks with 300B+ parameter MoE models (i.e. deep research, coding, etc..)
- **Steps:**
 - **Build Evals + Envs**
 - **Define the Task + Reward**
 - **Collect the Data**
 - **Train**
 - **Validate at small scale**
 - Run overfitting experiments
 - Validate the async RL pipeline
 - Ablate on hyperparameters
 - **Train the final model**
 - Iterate on final recipe

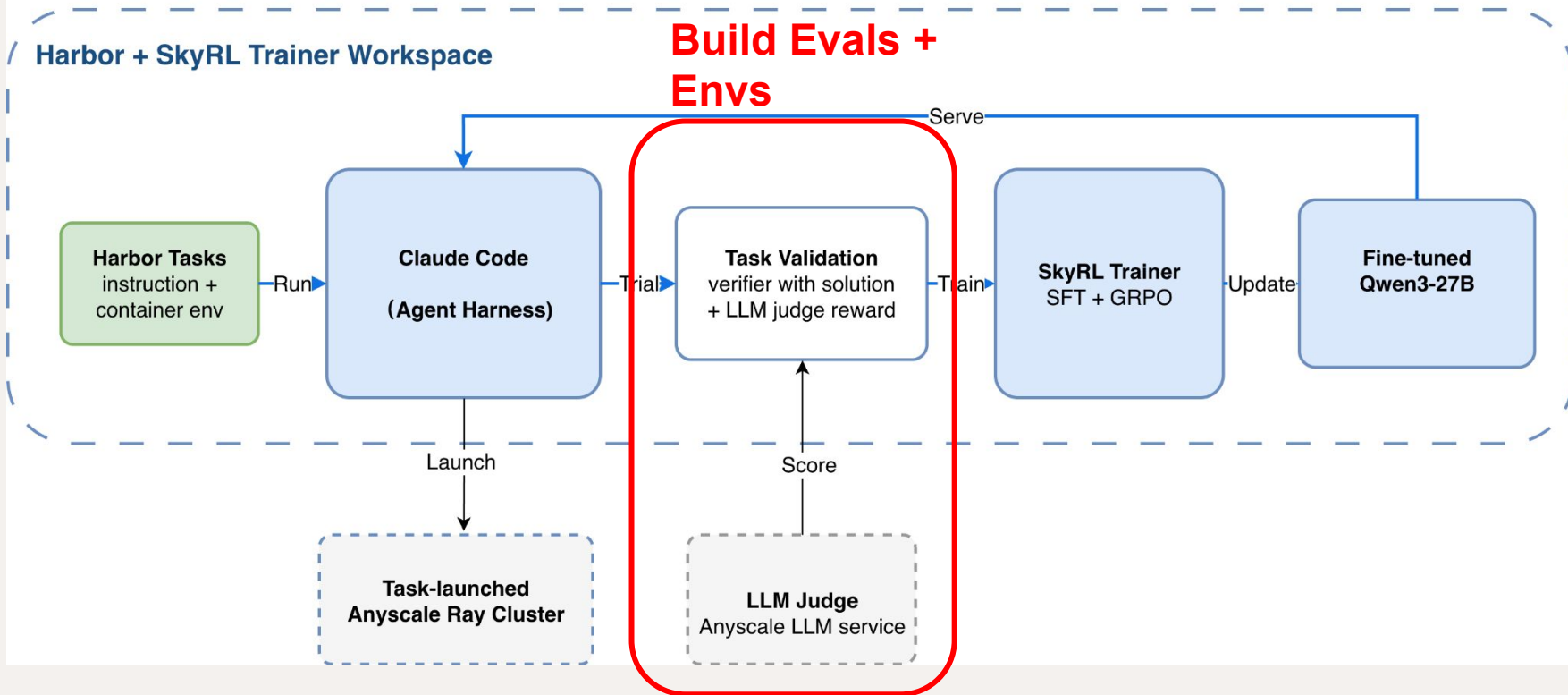
RL Challenges

- **Goal:** Training with multi-turn (20+), long context (64K+) agentic tasks with 300B+ parameter MoE models (i.e. deep research, coding, etc..)
- **Steps:**
 - **Build Evals + Envs**
 - **Define the Task + Reward**
 - **What judge to use? How to design rubrics? Float vs binary rewards?**
 - **Collect the Data**
 - **How can the data be collected/scaled? How do you want to split train/eval?**
 - **Train**
 - **Validation at small scale**
 - **Can we successfully overfit + hill climb with a weaker model? Are there small models that can provide enough reward signal?**
 - **Large model training**
 - **Does the small model behavior match larger models? Do hyperparameters transfer? Does inference become KV-cache bottlenecked?**

Fine-tune Qwen3 with SkyRL with Claude Code traces



Fine-tune Qwen3 with SkyRL with Claude Code traces



Building Evals + Envs

- Given a production coding trace - how do I turn it into an RL'able task?

Modern systems combine rewards rather than choosing one.

$$R = w_v R_{\text{verifier}} + w_r R_{\text{rubric}} + w_j R_{\text{judge}} + w_h R_{\text{human}}$$

VERIFIER

tests, exact
answers,
constraints

RUBRIC

create a checklist
for soft attributes

JUDGE

learned model
scores open-ended
quality

HUMAN

audits, escalations,
real outcomes

Building Evals + Envs

- Given a production coding trace - how do I turn it into an RL'able task?

Modern systems combine rewards rather than choosing one.

$$R = w_v R_{\text{verifier}} + w_r R_{\text{rubric}} + w_j R_{\text{judge}} + w_h R_{\text{human}}$$

VERIFIER

tests, exact
answers,
constraints

RUBRIC

create a checklist
for soft attributes

JUDGE

learned model
scores open-ended
quality

HUMAN

audits, escalations,
real outcomes

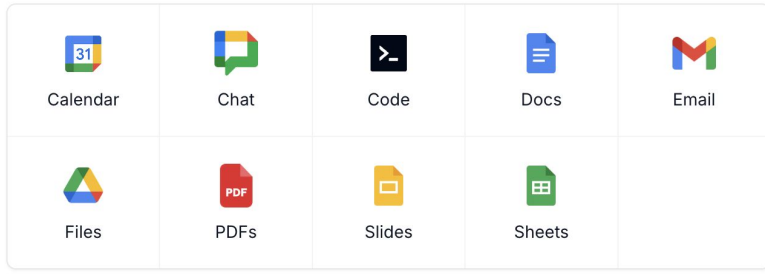
Rubrics + LLM as a Judge - Example

User Request

We need to review Summit's historical distributions. Write me a memo analyzing whether Summit's shareholder distributions are in accordance with US tax code. Cite the exact tax code in each instance (short citations are acceptable). Reply to me right in here please.

System Prompt

You are an agent that completes tasks independently. Use the tools provided to you to complete the task to the best of your ability.



Rubrics + LLM as a Judge - Example

Response

Final answer

MEMORANDUM

TO: Interested Parties **FROM:** Legal/Tax Analysis **DATE:** November 19, 2025 **RE:** Analysis of Summit Filing Solutions, Inc. Historical Shareholder Distributions for U.S. Tax Code Compliance

I. Executive Summary

After comprehensive review of Summit Filing Solutions, Inc.'s ("Summit") historical distributions from 2019-2024, significant compliance issues have been identified regarding shareholder distribution allocations under Subchapter S of the Internal Revenue Code. While Summit's S-corporation election and general structure appear compliant, the 2022 and 2024 distributions violate fundamental requirements for pro rata allocations to shareholders.

II. Background

Summit Filing Solutions, Inc. is a

User Request

We need to review Summit's historical distributions. Write me a memo analyzing whether Summit's shareholder distributions are in accordance with US tax code. Cite the exact tax code in each instance (short citations are acceptable). Reply to me right in here please.

System Prompt

You are an agent that completes tasks independently. Use the tools provided to you to complete the task to the best of your ability.

LLM

Criterion

States that Summit's shareholder distributions are disproportionate

States that Summit's disproportionate distributions violate US tax code

States that a nonresident alien receiving shares violates tax code

LLM Judge
(i.e.
Gemini,
GPT)

Rubric Grading

Reward

Score

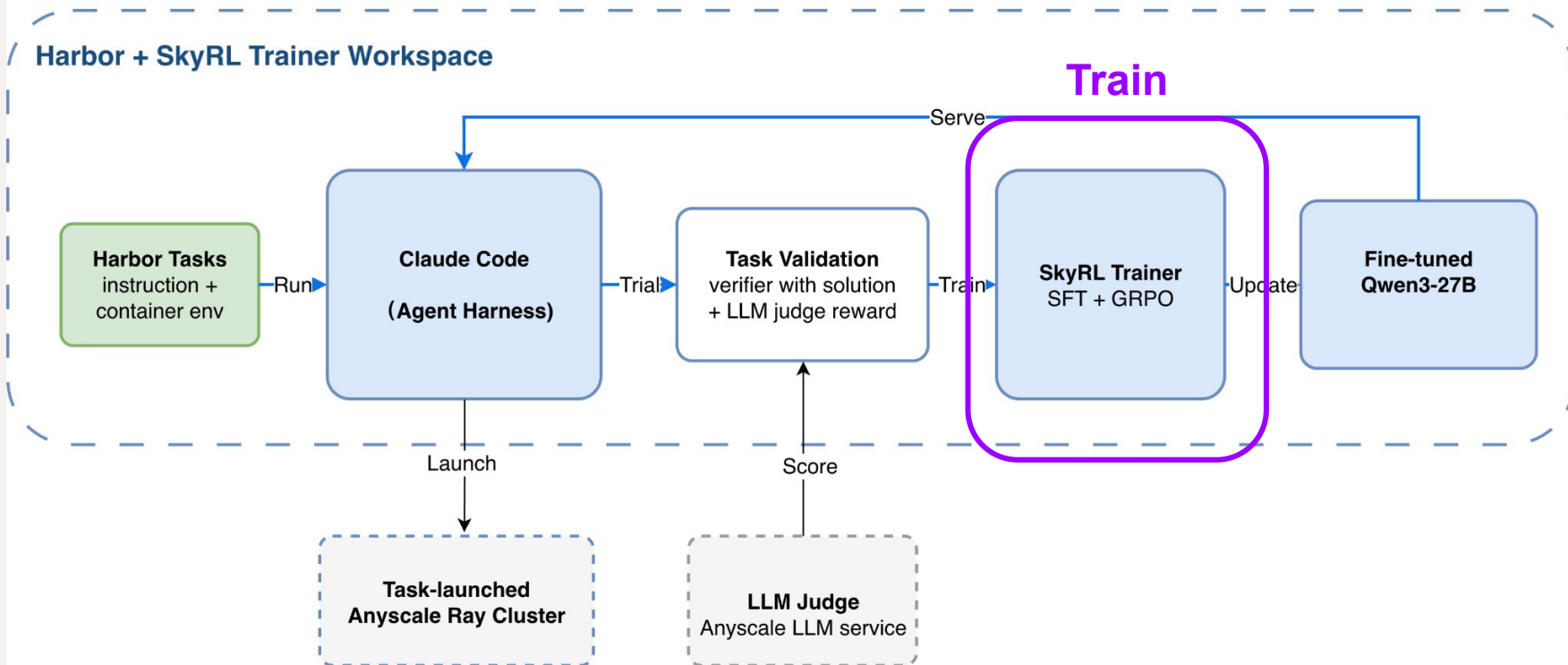
No

No

Yes

0.33

Fine-tune Qwen3 with SkyRL with Claude Code traces



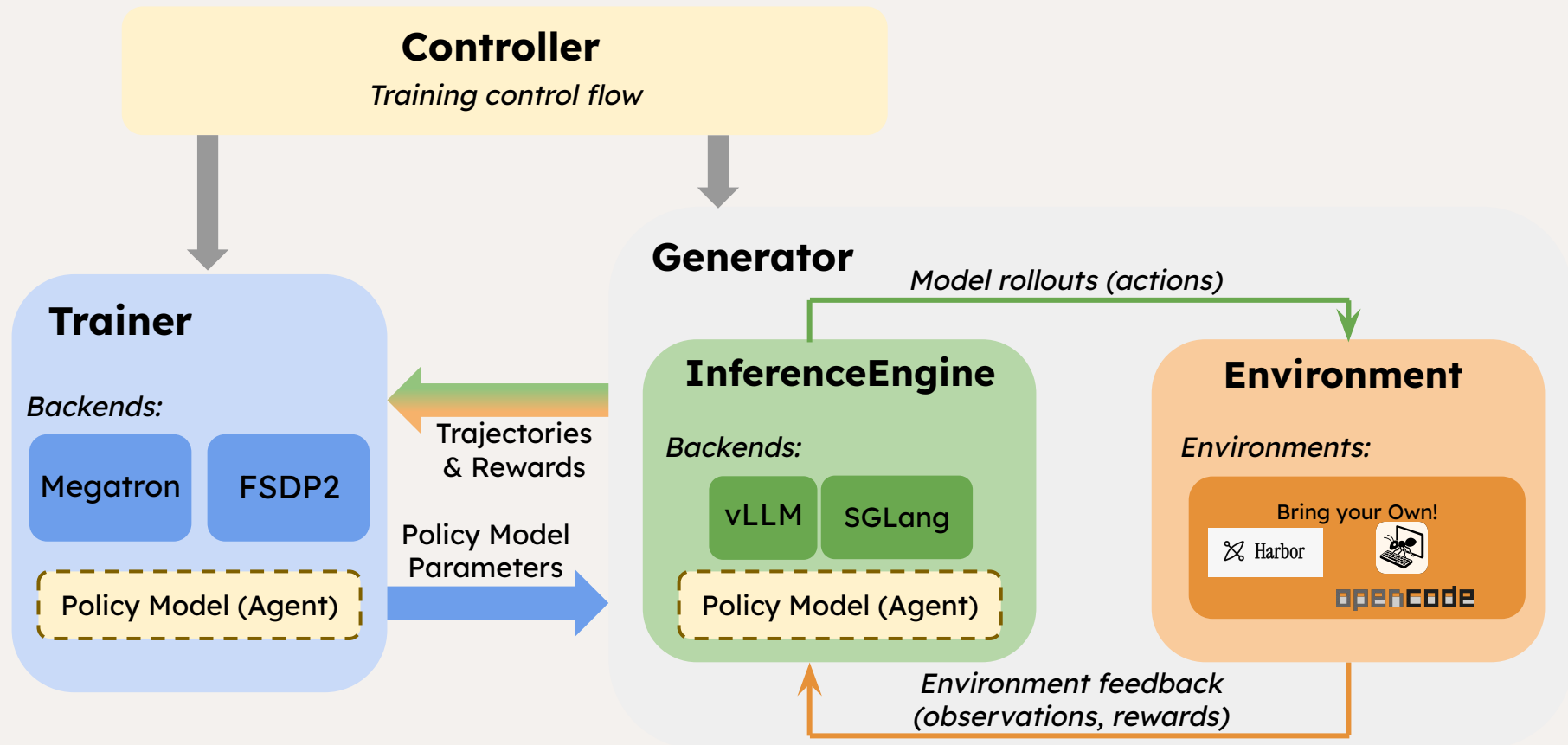
RL Systems

Why Ray for RL?

Nearly every open-source RL framework is built on Ray

Framework	Training engine	Serving engine	Orchestrator
TRL (Hugging Face)	Hugging Face Trainer	Hugging Face, vLLM	—
Verl (ByteDance)	FSDP, DeepSpeed, Megatron	vLLM, SGLang	Ray
OpenRLHF	DeepSpeed	Hugging Face, vLLM	Ray
AReal (Ant Group)	DeepSpeed, Megatron	vLLM, SGLang	Ray
Prime RL	FSDP	vLLM	—
ROLL (Alibaba)	DeepSpeed, Megatron	vLLM, SGLang	Ray
NeMo-RL (Nvidia)	FSDP, Megatron	vLLM	Ray
SkyRL (UC Berkeley)	FSDP, Megatron	vLLM, SGLang	Ray
SLIME (Z.ai)	Megatron	SGLang	Ray

RL Framework Architecture Diagram (SkyRL)



Ray for RL

Ray Python API



Tasks: stateless functions,
distributed across a cluster



Actors: stateful classes,
distributed across a cluster

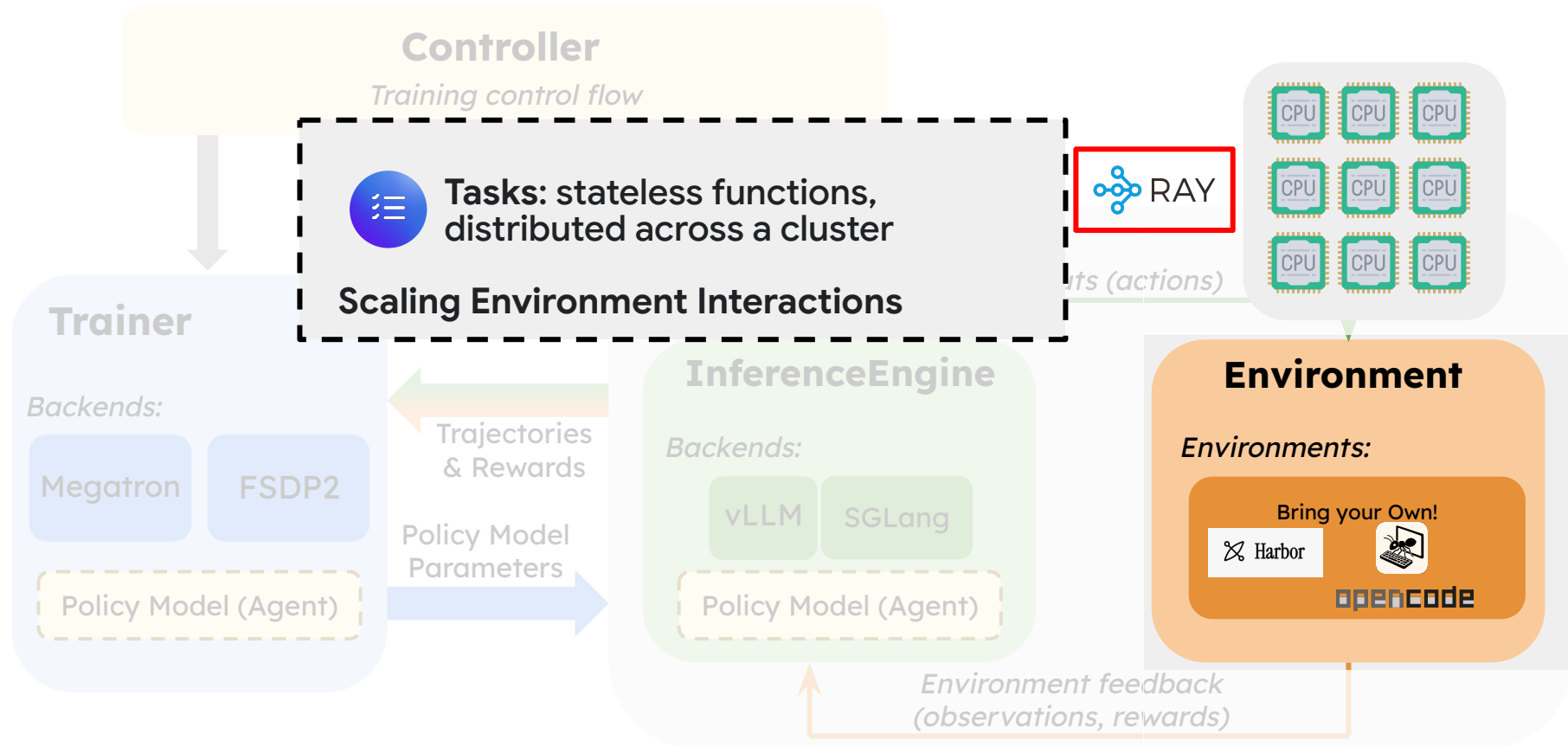


Objects: distributed futures,
high-performance object store

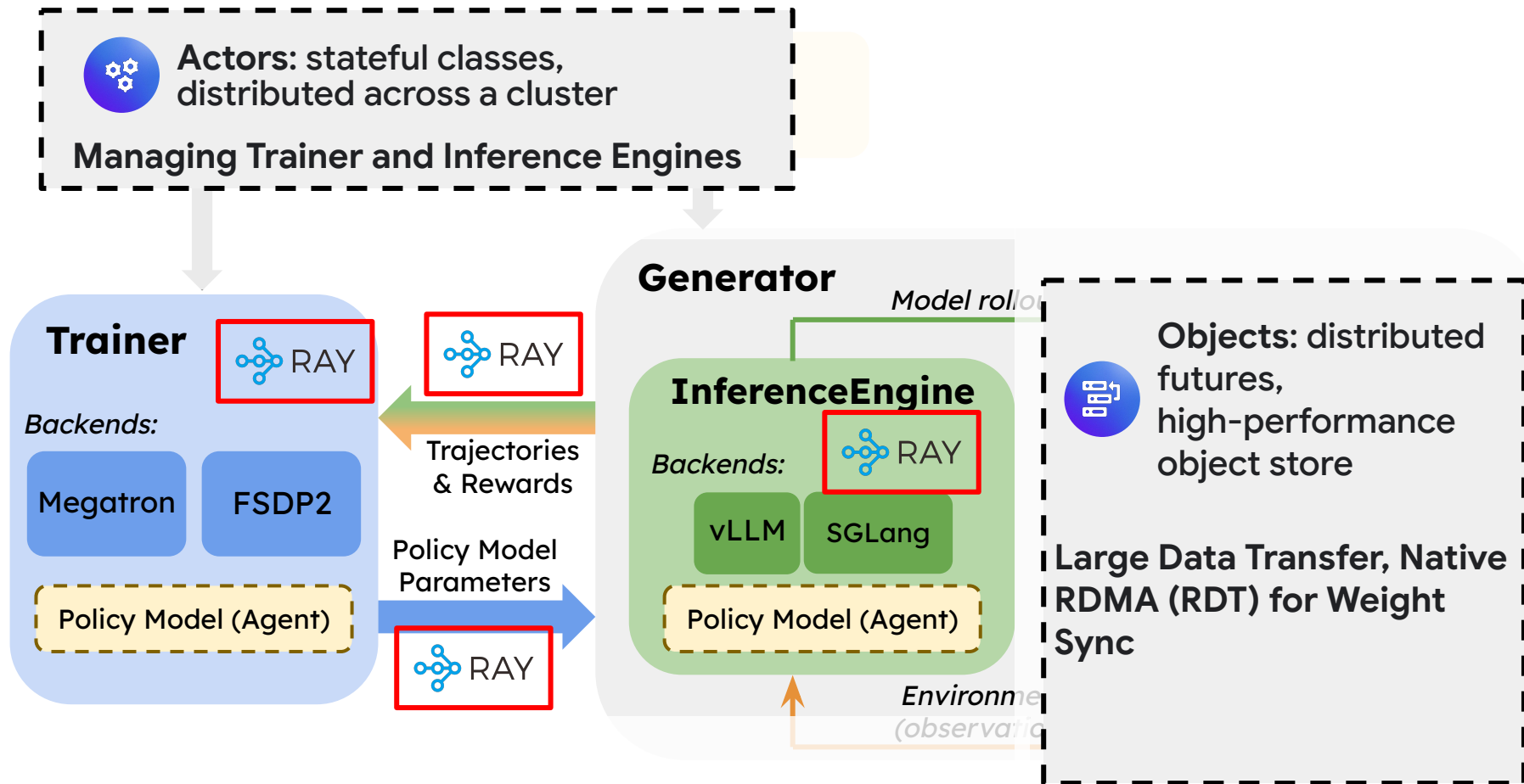


Resources: heterogeneous and
dynamic hardware requirements

Ray for RL



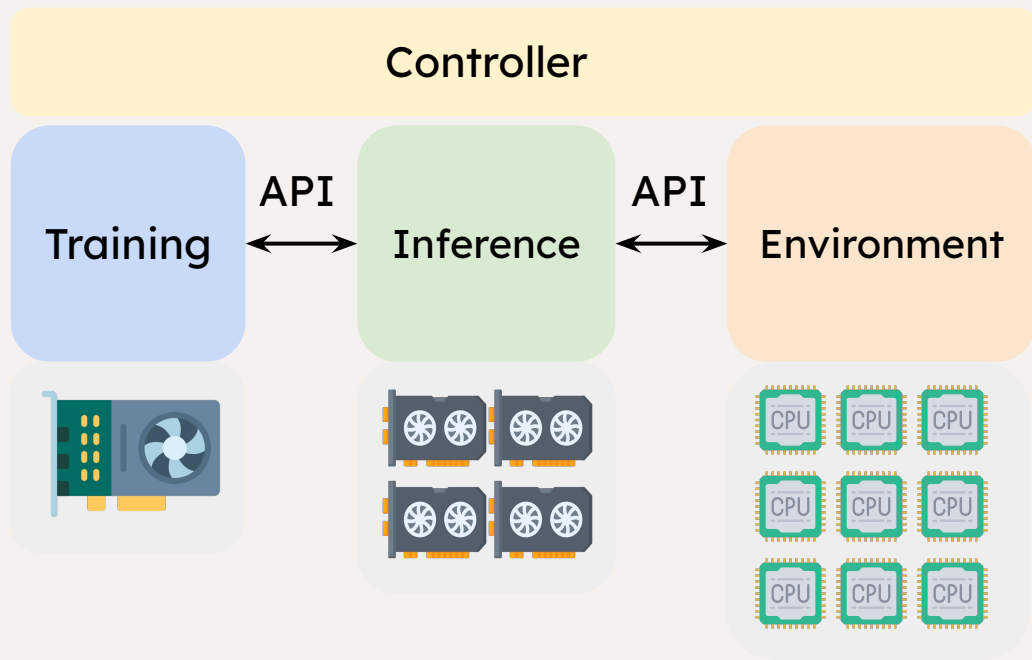
Ray for RL



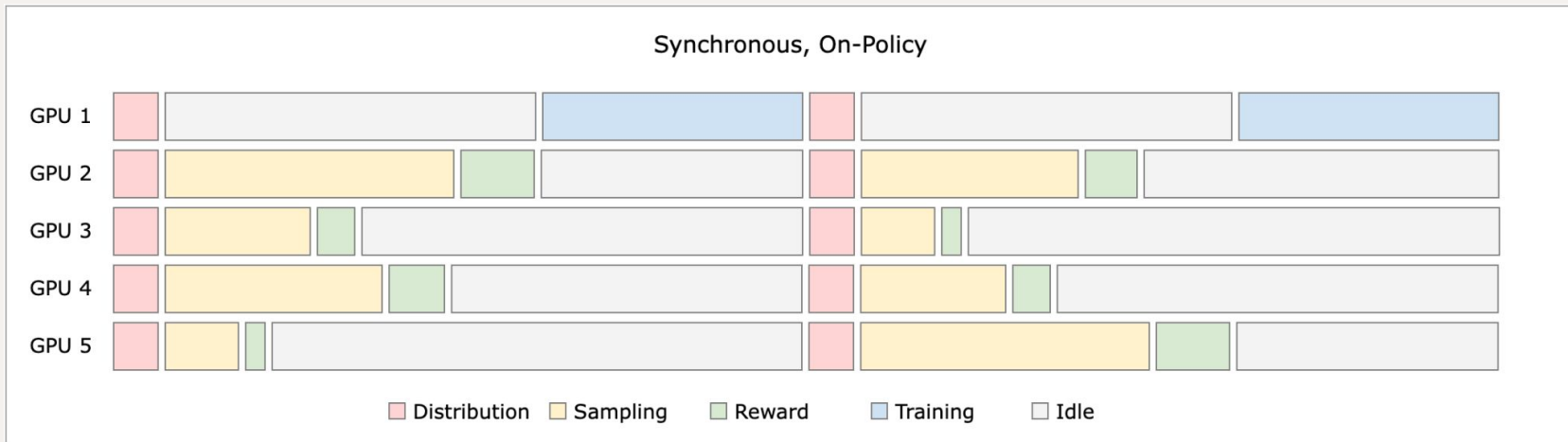
Ray for RL



Resources: heterogeneous and dynamic hardware requirements

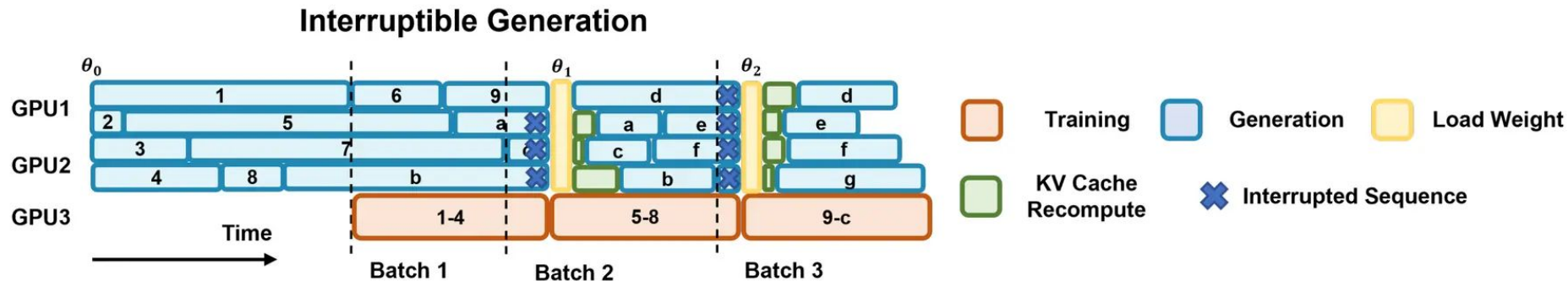


The problem with On-Policy RL



- Long context rollouts are bounded by the long tail of rollout generation
- Staying on-policy means lots of GPU idle time

Solution: Fully Async RL



- Each trajectory can be generated by up to N different versions of the policy
- Where N is the max number of off-policy steps

Demo Walkthrough: Fully Async RL with SkyRL

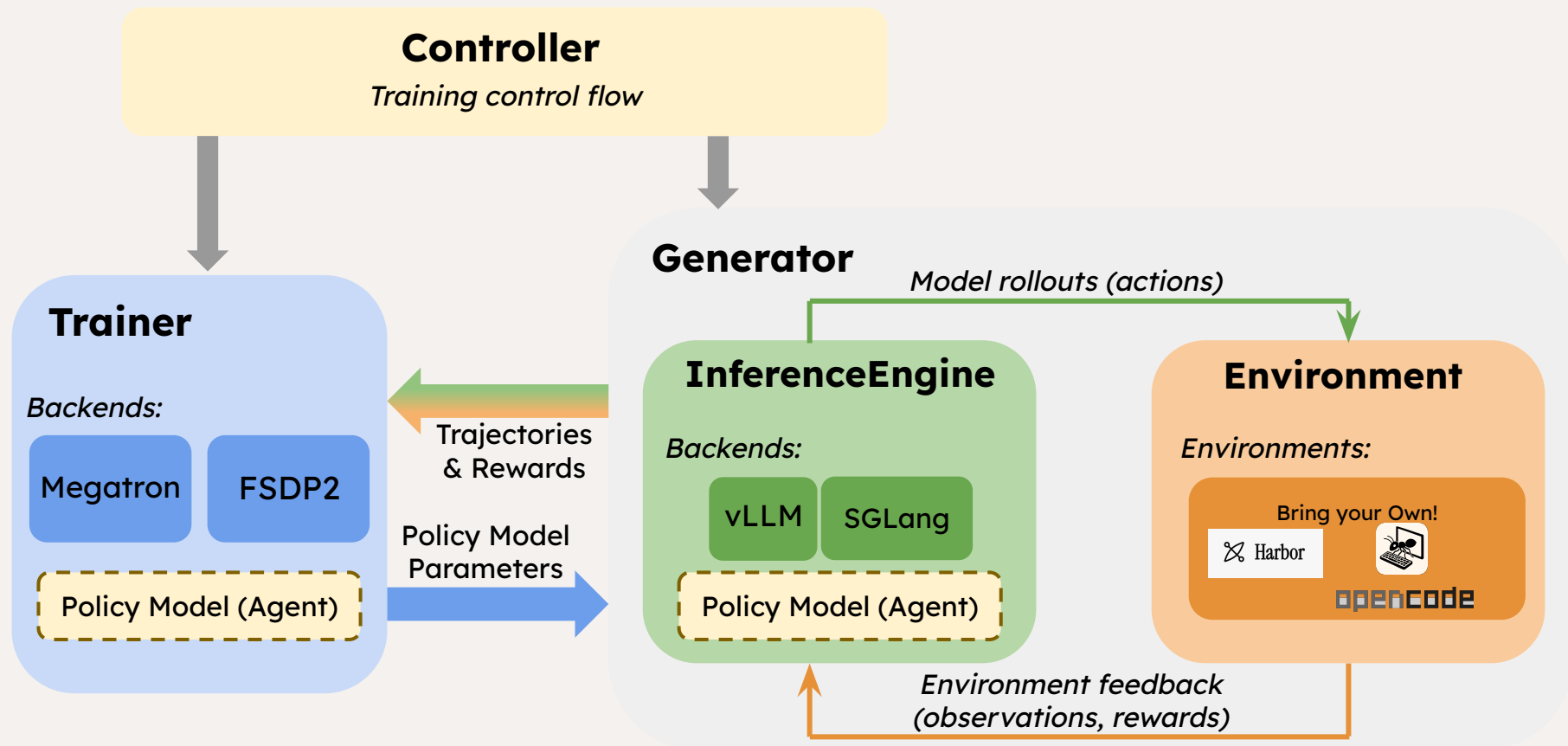
Reminder: Demo prompt

/anyscale-workload-llm-post-training can you git clone <https://github.com/NovaSky-AI/SkyRL>, and run the script `examples/train/fully_async/fully_async_run_gsm8k.sh` in it on the workspace <WORKSPACE NAME HERE> but using train batch size 64 and running training for just 1 epoch?

First, download the qwen2.5 model checkpoint from the s3 path `s3://llm-guide-use2/qwen2.5-1.5b/` to `/mnt/cluster_storage`. Then download the data files from `s3://llm-guide-use2/gsm8k/` to `/mnt/cluster_storage` as well.

Update the script to use the new paths for model and dataset at `/mnt/cluster_storage` and then use the console as the logger instead of wandb if WandB is not available.

SkyRL Architecture Diagram



Example - GSM8K: Dataset Preparation

Format Requirements

Each dataset entry is a dictionary with the following required (and some optional) values:

```
data = {
    "data_source": data_source,      # String: Name/identifier of the data source
    "prompt": [                     # List: Conversation format
        {
            "role": "user",
            "content": question,
        }
    ],
    "env_class": env_class,          # String: Environment class identifier
    "reward_spec": {
        "method": "rule",           # String: Either "rule" or "reward_model"
        "ground_truth": solution,  # Expected solution
    },
    "extra_info": {                 # Dict: Optional additional metadata
        # ... add your own fields here
    },
}
```

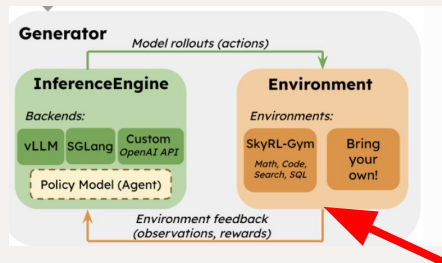
We load the dataset as a huggingface [DatasetDict](#).

Expected format

```
{
  'data_source': 'openai/gsm8k',
  'prompt': [
    {
      'content': 'Natalia sold clips to 48 of her friends in April,
                  and then she sold half as many clips in May. How
                  many clips did Natalia sell altogether in April
                  and May? Let\'s think step by step and output the
                  final answer after "####."',
      'role': 'user'
    }
  ],
  'env_class': 'gsm8k',
  'reward_spec': {'ground_truth': '72', 'method': 'rule'},
  'extra_info': {
    ...
  }
}
```

GSM8K example

Example - GSM8K: Environment Definition



```

class GSM8kEnv(BaseTextEnv):
    """
    Environment for Math execution tasks.
    """

    def __init__(self, env_config: DictConfig, extras: Dict[str, Any] = {}):
        super().__init__()

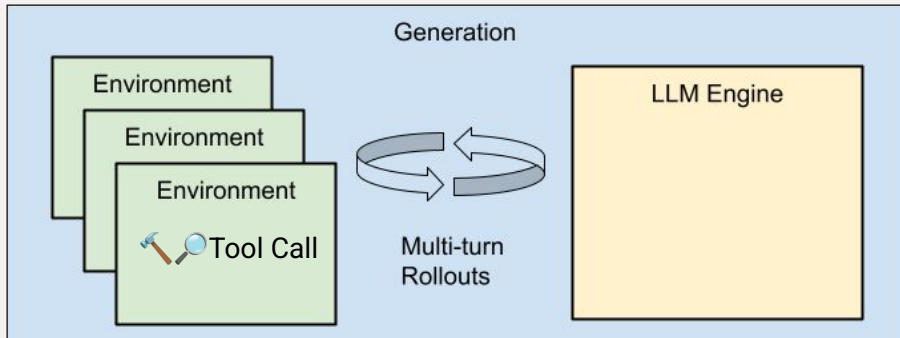
        assert "reward_spec" in extras, "reward_spec field is required"
        assert "ground_truth" in extras["reward_spec"], "ground_truth is required in reward_spec field"
        self.ground_truth = extras["reward_spec"]["ground_truth"]

    def _get_reward(self, action: str) -> float:
        return utils.compute_score(action, self.ground_truth)

    def step(self, action: str) -> BaseTextEnvStepOutput:
        done = True # always done after one step
        reward = self._get_reward(action)

        # No observation in gsm8k, and no tool call
        return BaseTextEnvStepOutput(observations=[], reward=reward, done=done, metadata={})
    
```

Example - Search: Multi-Turn RL + Tool Calling



Python

```
import skyrl_gym
```

```
env = skyrl_gym.make("searchEnv")
```

```
llm_input = env.init(question)
```

```
num_turns = 10
```

```
for _ in range(num_turns):
```

```
    action = llm.generate(llm_input)
```

```
    obs, reward, done = env.step(action)
```

```
    # update LLM input with new action and resulting observation
```

```
    llm_input.update(action, obs)
```

```
    if done:
```

```
        break
```

```
env.close()
```

Example - Search: Dataset

Task Overview

In this task, the agent is given a natural language question and the ability to query a search engine. The agent can use the search engine to help answer the question. An example prompt is shown below:

You are a helpful and harmless assistant.

Answer the given question. You must conduct reasoning inside `<think>` and `</think>` first every time you get new information. After reasoning, if you find you lack some knowledge, you can call a search engine by `<search> query </search>` and it will return the top searched results between `<information>` and `</information>`. You can search as many times as you want. If you find no further external knowledge needed, you can directly provide the answer inside `<answer>` and `</answer>`, without detailed illustrations. For example, `<answer> Beijing </answer>`.

Question: In what year was the company that was founded as Sound of Music added to the S&P 500?

Answer: Best Buy in **1999**

Example - Search: Environment Definition

```
class SearchEnv(BaseTextEnv):
    """
    Environment for Search execution tasks.
    """

    def __init__(self, env_config: DictConfig, extras: Dict[str, Any] = {}):
        super().__init__()

        # Initialize the tools
        # name is hardcoded to "SearchToolGroup", with tool name "search"
        self.tool_group = SearchToolGroup(
            search_url=env_config.search_url,
            topk=env_config.topk,
            timeout=env_config.timeout,
            log_requests=env_config.log_requests,
        )
        self.init_tool_groups([self.tool_group])

        # Chat history
        # role (user, assistant), content (tool observation or LLM response)
        self.chat_history: ConversationType = []
```

```
class SearchToolGroup(ToolGroup):
    @tool
    def search(self, query: str) -> str:
        query = query.strip()

        try:
            api_response, error_msg = call_search_api(
                retrieval_service_url=self.search_url,
                query=query,
                topk=self.topk,
                timeout=self.timeout,
                log_requests=self.log_requests,
                session=self.session, # Pass our shared session for connection reuse
            )
```

Example - Search: Environment Definition

```
def step(self, action: str) -> BaseTextEnvStepOutput:
    self.turns += 1
    action = self._postprocess_action(action)
    self.chat_history.append({"role": "assistant", "content": action})

    done = self._is_done(action)
    reward = self._get_reward(action, done)

    if done:
        return BaseTextEnvStepOutput(
            observations=[], reward=reward, done=done, metadata={}, postprocessed_action=action
        )

    query = self._parse_action(action)
    observation = self.execute_tool("SearchToolGroup", "search", query)
    new_obs = {"role": "user", "content": observation}
    self.chat_history.append(new_obs)

    return BaseTextEnvStepOutput(
        observations=[new_obs] if new_obs else [],
        reward=reward,
        done=done,
        metadata={},
        postprocessed_action=action,
    )
```

Compute reward for trajectory (is answer right?)

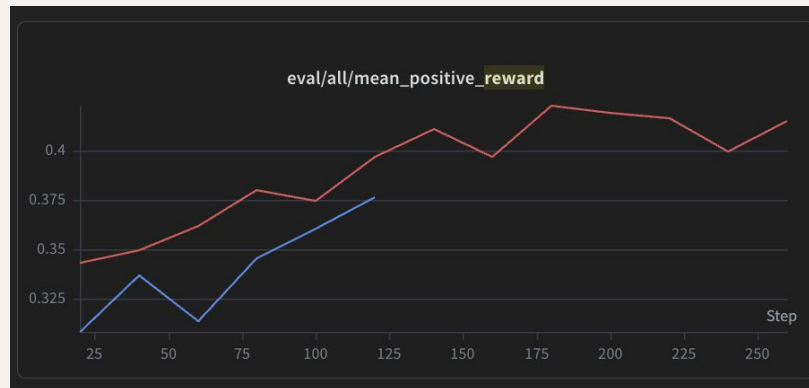
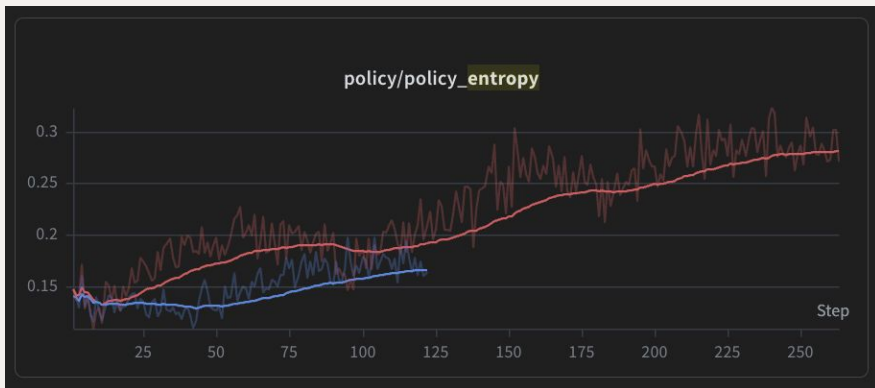
```
def _parse_action(self, action: str) -> List[Optional[str]]:
    match = None
    if "<search>" in action and "</search>" in action:
        match = re.search(r"<search>(.*?)</search>", action, re.DOTALL)
    return [match.group(1)] if match else [None]
```

 Tool Call

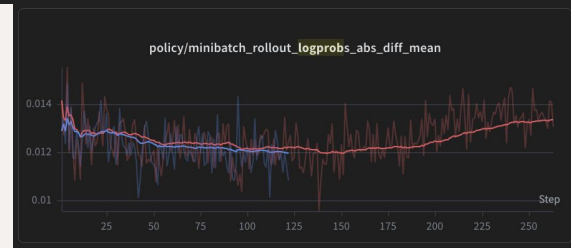
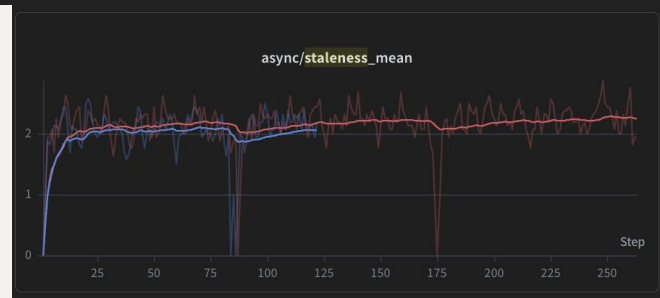
Return search output to generator loop

Observability walkthrough on a SkyRL job

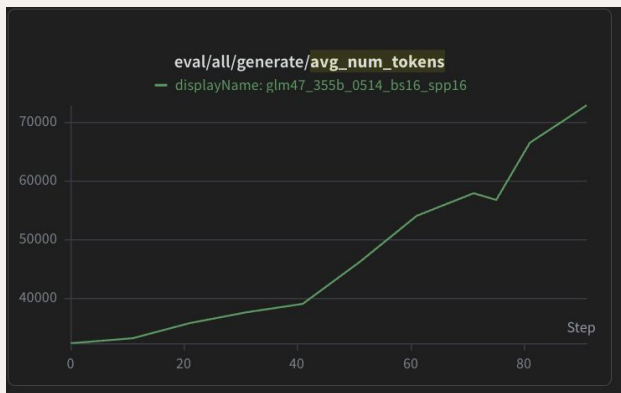
Key RL Metrics - Training



- **Entropy:** Should stay stable or grow
 - Can use entropy aware loss, or top-p sampling
- **Staleness:** Minimize the policy staleness
 - Tune # inference vs # train GPUs
- **Logprob diff:** Make sure this doesn't grow
 - Can adopt router replay for MoE
- **Reward:** 



Key RL Metrics - Inference



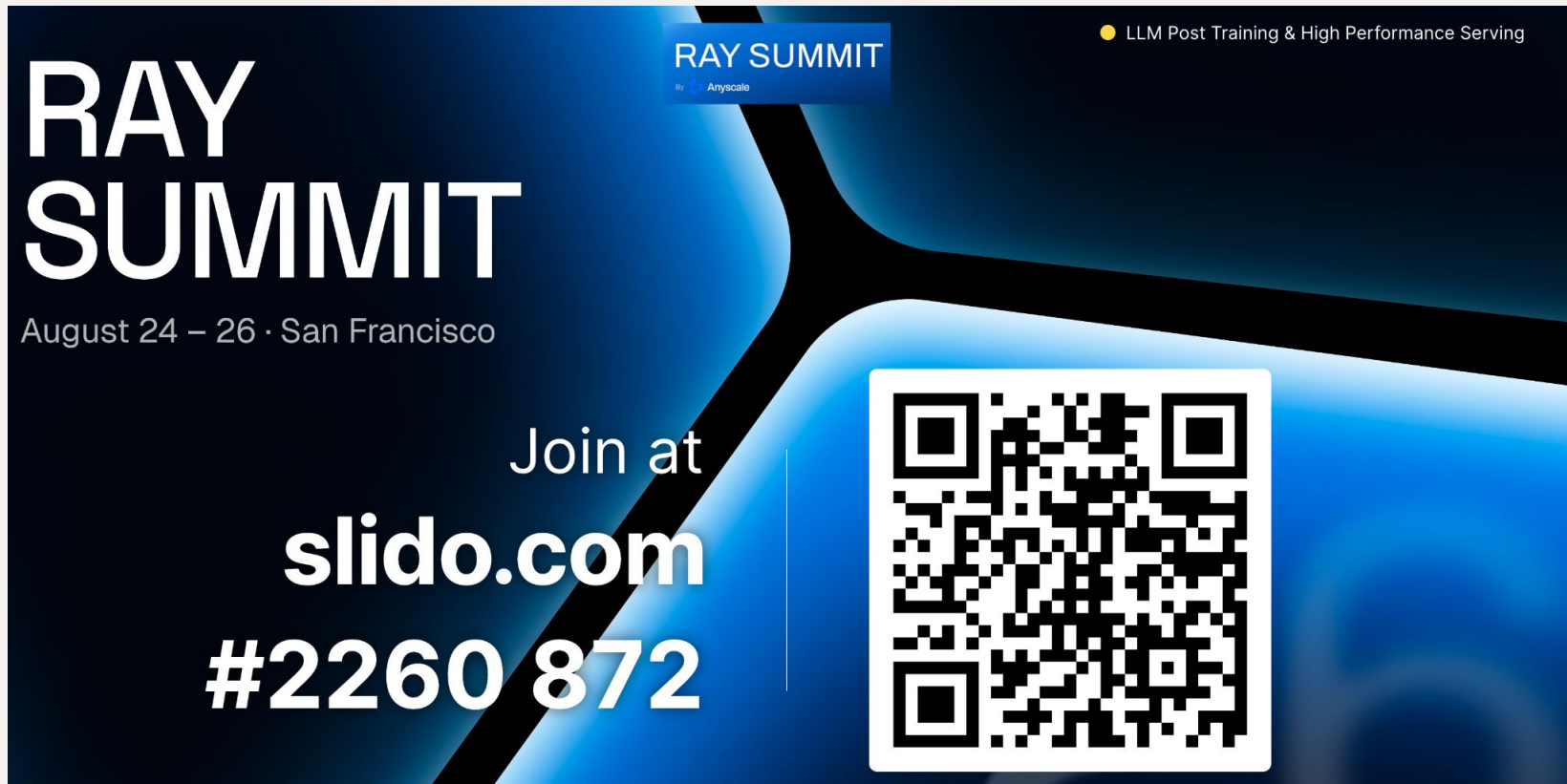
- **Generation length**
 - Will often blow up during training
 - Can try using length penalties
- **Inference throughput**
 - Observability is very important
 - KV cache misses are **very** expensive in long context multi-turn settings
- **Unexpected failures**
 - Trajectory/Failure visualization is very important
 - Sandbox robustness and error/timeout observability



Add vLLM Metrics



<https://gist.github.com/erictang000/9fb2e7e3342df567a83da54f6a4099cc>



**RAY
SUMMIT**

August 24 – 26 · San Francisco

Join at
slido.com
#2260 872

RAY SUMMIT
By Anyscale

● LLM Post Training & High Performance Serving



Thank You & Next Steps



Try Anyscale

- Deploy your existing Ray code without any modifications
- Measure real performance gains on your specific use cases
- Evaluate our enterprise features with your own data, in your own VPC

**Try it today with
\$100 in free credits**



Get Expert Training

- Get **hands-on training** and expert support from the team that created Ray.
- Training reimaged - Developed for Engineers and Delivered by Engineers
- Hands on and oriented to make engineers productive



Join the Community



Connect with other practitioners on **Rau Slack**



Ask questions and share learnings on our **forum**



Become a contributor and follow along on our **GitHub**

Survey & Feedback



https://docs.google.com/forms/d/e/1FAIpQLSdFjWA1f1edoM4cU2jQFjbOdmi_gp9yEKGBrKGuPOgpDTly9g/viewform